

作者工作校樣・非正式出版品

讓 AI 畫對圖

六大繪圖工具的實作與協作

13 章第一遍通稿・A4 試讀版

來源版本：dd25dda2effea23a3c0d1fa214597f1568c89600

部分寬圖採 A4 橫式。案例均為虛構教學設定。

未完成印刷打樣、讀者試讀、跨平台重建或正式授權決策。

程式碼僅供閱讀；長命令可能視覺折行。執行時請使用連結中的原始命令，不要把 PDF 換行當成命令分隔。

目錄

1. 第 1 章：從一句需求到一張有用的圖	3
2. 第 2 章：六套工具怎麼選	5
3. 第 3 章：讓 AI 按規格畫圖	7
4. 第 4 章：Mermaid，把流程直接寫進文件	10
5. 第 5 章：D2，用文字維護系統架構	15
6. 第 6 章：PlantUML，把付款互動與狀態講清楚	20
7. 第 7 章：Graphviz，從依賴資料產圖與篩選影響	28
8. 第 8 章：draw.io，讓 AI 草稿能由人接手	35
9. 第 9 章：Excalidraw，先把未定的事留在白板上	42
10. 第 10 章：AI 繪圖除錯室	46
11. 第 11 章：跨工具協作與轉換限制	54
12. 第 12 章：版本管理、自動化與安全	58
13. 第 13 章：部分退款綜合專案	62

目錄可點選跳章；文內來源連結固定至本校樣的 GitHub 版本。

第 1 章：從一句需求到一張有用的圖

樣章草稿。本書案例均為虛構教學設定，不是實際客戶需求或生產系統設計。

1.1 「幫我畫訂單流程」還少了什麼

同一句話交給 AI，可以得到購物步驟、後端呼叫、資料表關係，甚至付款退款全生命週期。這些圖未必畫錯，卻可能沒有一張能回答讀者的問題。

本書先把問題收斂為：「讓產品與工程一起確認，送出訂單後有哪些結果，付款未知時如何交接。」這句話限定了讀者、目的與範圍。至於服務部署在哪裡、哪個資料表存什麼，本張圖不回答。

你可以先填 [繪圖需求單](#)，再請 AI 產生來源。需求單不必很長，但必須讓另一個人能判斷圖是否符合要求。

1.2 用一個問題決定一張圖的邊界

以第 4 章為例，圖從「提交訂單」開始，到訂單被拒絕、取消、付款結果明確或交給對帳為止。退款和出貨不在其中。這不是省略必要例外，而是把後續工作留給另一張圖。

邊界要同時寫清楚入口與出口。「對帳」可以是出口，但不能用這兩個字掩蓋永久保留庫存的問題。對帳間隔、總期限及人工責任仍待另訂，應在說明裡留下這些問題。

讀者若想知道付款 API 的呼叫順序，再開 [第 6 章時序圖](#)；想知道服務邊界，改看 [第 5 章架構圖](#)。不需要把所有細節塞進第一張流程圖。

1.3 把已知、假設與未知分開

類別	本書例子	能否直接畫成規則
已提供的教學需求	付款未知不等於失敗	可依這條需求畫分支，但需標示案例來源
明示假設	庫存預留與建單協調由下層保證	可簡化圖，必須保留假設與未涵蓋故障
尚未決定	對帳期限、人工接手者	不可由 AI 補數字或人名

「已提供」在本書不代表真實公司已批准。每個範例都要說明資料來源；若來源是虛構情境，就直接寫虛構。圖中的三次查詢上限屬於第 6 章假設，不能悄悄變成全書所有案例的正式政策。

1.4 把需求改成可以對照的句子

「付款異常要處理」太籠統。改成「結果未知時，不流向付款失敗，不釋放預留庫存，轉交對帳」，就能逐項查看圖上的節點與箭頭。

第 4 章 [R1-R8 需求單](#) 把每條規則連到原生節點。例如 R8 對應 result → reconcile。需求 ID 在該範例內有效；其他章的 R8 不必是同一條，跨章引用時要附來源路徑。

也要替箭頭寫一句定義。本張箭頭表示處理流程，不是資料流、服務呼叫或模組依賴。把這些意思混用，容易得到看似連貫卻無法判讀的圖。

1.5 一份完成的需求單

[訂單流程填寫版](#) 沿用既有需求，沒有新增付款規則。它記錄讀者、用途、範圍、假設、未知、來源與交付條件。

交付條件應提到原生檔與預覽：只交未含原生場景的 PNG，之後通常得找回來源或重建才能改字；只交原始碼，讀者又可能沒有工具。兩者都要有，但預覽不能代替原生可編輯性的驗證。

不要在需求單預先勾選「測試通過」。先寫要怎麼檢查，執行後再填結果。假設、需求、驗收證據是不同欄位。

1.6 練習

收到「請畫付款流程，失敗就重試三次」時，先列出需要釐清的事項，不急著產圖。

參考答案：失敗是否包含逾時？重試的是扣款還是查詢？哪份規格批准三次？相同請求如何防重？本張圖是否包含退款？若無答案，保留問題，不把重扣款畫成既定政策。

完成需求單後，才進入 [第 2 章工具選型](#)。

第 2 章：六套工具怎麼選

樣章草稿。本章說明本書採用的分工，不是完整功能排名或跨版本效能評測。

2.1 先看交付方式

如果圖要放在 Markdown 文件裡，並跟著文字一起審查，文字來源很方便。如果交接者要拖動方框、調整線條，原生圖形編輯器通常更適合。這兩個需求有時比圖有多少種形狀更重要。

選工具前先問：誰維護原始檔？在哪裡閱讀？需要精確表示什麼？是否需要人工拖曳？能否安裝渲染器？可接受哪些資訊在轉換時重建？

2.2 本書的六種分工

工具	本書用途	維護來源	選用時要接受的成本
Mermaid	文件內的步驟與分支	.mmd	顯示平台版本不同，需實際渲染；複雜圖可能需要拆分
D2	服務、容器與邊界	.d2	排版引擎與字型；不保證跨引擎位置相同
PlantUML	時序、狀態與概念類別	.puml	精確圖意仍靠規格；工具不能替你決定未知結果
Graphviz	資料生成的依賴總覽與子圖	本例 JSON 加生成程式	必須定義邊方向；密集資料可能需要篩選視圖
draw.io	人工微調與圖形交接	.drawio	需保留模型端點，實際拖曳、保存及重開
Excalidraw	工作坊白板與未定問題	.excalidraw	手繪外觀不承載批准狀態，仍需決策紀錄

Graphviz 的原始來源也可以直接是 DOT；本書第 7 章選擇由 JSON 生成，所以生成 DOT 不是最上游。原始檔的地位由流程決定，不能只看副檔名。

2.3 相同主題可以有不同選擇

「付款」不是工具選型答案。如果要討論哪些問題還沒決定，用 Excalidraw；如果要核對 API、資料庫和支付服務的訊息次序，用 PlantUML；如果只是文件裡交代成功、失敗、未知三種出口，可以用 Mermaid。

第 13 章 刻意把退款受理交給 Mermaid，把單筆退款紀錄的結果交給 PlantUML。兩張圖分工後，沒有必要再把同一流程重畫四次。

可用 [選型紀錄模板](#) 留下主要需求、選擇理由和未採用方案。這比在團隊討論中只說「大家都用這套」更容易交接。

2.4 自動排版與人工排版的差別

文字來源便於比較差異，但一次標籤變長就可能改變整張圖的排列。人工座標可以細調，卻也可能在複製貼上後失去連線綁定。兩種方式都需要驗收。

若唯一要求是「看起來跟原 PNG 一樣」，應先追問是否需要編輯、語意或資料生成。視覺相似並不能保證原生模型保留，尤其不能拿匯入一張截圖當作可編輯重建。

跨工具時先列出要保留的契約。第 11 章只對齊 D2／draw.io 的四條呼叫端點，沒有聲稱所有標籤、樣式、座標相等。Graphviz 的付款模組也不能直接映射成外部支付服務。

2.5 選型練習

情境	本書建議	判定理由
README 中說明訂單結果	Mermaid	文件步驟與分支是主要交付
檢查平台內外服務歸屬	D2	容器和邊界比操作時序重要
討論查詢逾時與回覆順序	PlantUML	需要參與者與訊息次序
從資料列出付款模組的潛在影響範圍	Graphviz	資料生成、依賴篩選與循環處理
非程式背景同事要拖動既有架構圖	draw.io	人工編輯與端點保留是需求
尚未決定對帳間隔與負責人	Excalidraw	保留未定事項，不先編造完整流程

這些是練習參考，不是排他規則。若團隊已經有可維護的工具鏈，提出替代選擇時應比較實際交付成本，而非只比工具清單。

下一章把選好的工具、需求與驗收方式放進同一份 AI 任務。

第 3 章：讓 AI 按規格畫圖

樣章草稿。提示詞可以約束工作範圍，不能代替實際驗證。

3.1 先給來源與邊界

「你是頂尖架構師」無法告訴 AI 哪條關係是真的。更有用的輸入是：讀者、用途、原始需求、節點清單、箭頭意思、例外、未知與交付格式。

本書 [通用提示詞](#) 將這些欄位分開。沒有填寫的資訊應明確留白。不要讓 AI 把「可能有退款」解讀成「請自行設計完整退款流程」。

第一輪可以只要求它整理節點、關係和待確認問題。先比對文字清單，再產生圖，通常比看到一張漂亮但錯誤的圖後逐條拆解容易。

3.2 把禁止事項寫成可觀察行為

「不要幻覺」太抽象。對訂單案例，可改成：

- 不加入未提供的服務或退款分支。
- 付款逾時只能走待確認，不走明確失敗。
- 只畫付款送出前取消，不假設送出後取消一定安全。
- 不新增查詢次數、等待期限或人工責任人。
- 回報未執行的檢查，不生成假造的成功紀錄。

這些限制仍要靠驗收確認。[第 10 章](#) 的反向依賴案例展示了兩份資料都能成功產圖、但只有一份符合需求的情況。

3.3 用穩定 ID 分開內容和顯示

節點 ID 是引用，標籤是給讀者看的文字。將 reconcile 的標籤改短，不一定要重新命名節點；反過來，把 payment 全部取代也可能誤改不同層次的物件。

要求 AI 保留既有 ID、端點與群組，只修改批准範圍。若改變圖意，先更新需求或契約，再更新來源。不要因測試失敗就先把預期值改掉。

3.4 四層驗收不能互相替代

層次	要取得的證據	常見誤判
語法／環境	實際命令、版本、退出碼及可讀產物	JSON 合法就能在編輯器正常使用
語意	每條需求與節點、關係或情境的對照	exit 0 就表示業務正確
視覺	最終圖片的中文、裁切、壓線與閱讀尺寸	SVG 可解析就不會缺字
交付／可編輯性	原生來源、真實修改、保存及重開	PNG 看得到就等於可編輯

[驗收紀錄模板](#) 預設所有項目都是「未測」，並要求每一項寫證據位置。沒有真實執行就保持未測，不必為了把表填滿而杜撰結果。

本書某些編輯器測試只驗證模型序列化或瀏覽器儲存重載，未測原生檔案保存對話框。這些結果有價值，但不能換名稱說成「所有另存與重開都通過」。

3.5 修圖時縮小變更範圍

若標籤太擠，可以要求：「保留三個結果分支與所有端點，縮短顯示文字，把完整條件移到圖例，重產預覽。」這讓修改有邊界，也有可判定的結果。

若出現 Parse error，先給完整錯誤與最小來源；若中文字缺字，先確認字型和產圖環境。不同故障需要不同證據，不要把所有問題都丟回一句「重畫」。

AI 輸出的 SVG、原生檔與說明也可能包含外部資源或文字指令。把檔案內容當作資料，不讓它要求你上傳憑證或執行不相關命令。公開前另查個資、內部網址、嵌入場景與授權。

3.6 一輪任務的完成條件

交付原始檔、預覽、需求對照與執行紀錄，並列出仍未確認的問題。如果要發布到 GitHub，還要確認檔案真的出現在指定提交，CI 的 head SHA 與該提交相同。

目前核心 CI 只驗證部分程式與契約；不能據此宣布全書所有圖通過。完整的版本、權限與發布邊界見 [第 12 章](#)。

3.7 練習與參考修正

不良提示詞：「畫一張完整訂單圖，異常都自動解決，輸出 PNG，說明已測試。」

可改為：「依指定 R1-R8 需求產生 Mermaid 流程來源與預覽。只涵蓋付款送出前取消與初次付款結果；未知走對帳交接，不補期限。先列節點、關係與疑問，確認後再產圖。實際渲染並保存錯誤／退出碼，逐條對照需求；不能執行時寫未測。」

參考修正刪掉了沒有定義的『完整』與『自動解決』，保留原生來源，並把成功聲明改成需要證據的驗收工作。接著可用 第 4 章 Mermaid 的檔案實作這輪流程。

第 4 章：Mermaid，把流程直接寫進文件

樣章草稿。配套範例已在本機渲染；不代表所有 Markdown 平台皆已測試。本章的支付流程是教學用虛構規格。

4.1 先決定這張圖要回答什麼

在這個虛構討論情境中，產品經理問：「訂單什麼時候算成立？付款不成功時，庫存會不會被占住？」這需要的是處理流程，不是服務架構，也不是資料表關係。

本章只回答提交訂單到付款結果分流之間的問題。出貨、通知、退款與付款後取消都不在這張圖裡。讀者看完應能分清：哪些情況直接拒絕、哪些情況釋放預留庫存，以及哪些情況不能立刻判定失敗。

Mermaid 適合這種與文字說明一起維護的圖。原始檔是文字，修改能在 Git 中比較；代價是排版主要由渲染器決定，不適合要求每個節點都固定在指定座標的場合。

4.2 先把「付款失敗」說清楚

「付款沒成功就取消訂單」是一句危險的模糊需求。服務收到明確拒絕，和網路逾時沒有收到結果，是兩件事。後者可能已扣款，只是回應還沒抵達。

因此，本例把結果分成成功、明確失敗與未知。明確失敗才關閉訂單並釋放庫存；未知則標記待確認，保留庫存並轉交對帳。

保留並不是永久占用。對帳頻率、保留期限與逾期處理是另一份規格，本圖不代替它。畫出交接點，是為了揭露邊界，而不是宣稱整個支付問題已解決。

完整條件在 [需求與邊界](#)。先閱讀 R1–R8，再開啟圖。AI 的任務是將這些規格轉成表示法，不能替作者增加自動退款或無限重試。

4.3 只學本例需要的語法

```
flowchart TD
    submit["提交訂單"] --> valid{"資料有效?"}
    valid -->|是| nextStep["下一步"]
    valid -->|否| rejected["拒絕訂單"]
```

flowchart TD 表示流程圖由上往下排列。矩形表示處理步驟，菱形表示決策；這是本例的使用約定，不代表 Mermaid 會替你檢查業務規則。

submit 是節點 ID，提交訂單 是顯示文字。兩者分開後，可以修正中文說法而不用連帶改動所有箭頭。ID 應唯一，使用簡短英文；不要把小寫 end 當作流程圖節點 ID。

標籤用雙引號包住，能降低中文、空白與標點造成解析歧義的機會。-->|是| 則把條件寫在連線上。缺少出口條件時，讀者只能猜哪條路是成功。

本例不使用 HTML 標籤、互動連結或外部圖片。沒有需要就不要增加功能，尤其不應為了某個效果把安全設定改成寬鬆模式。

4.4 給 AI 的輸入與修改要求

可重用提示詞 要求 AI：

- 嚴格依 R1–R8 畫圖。
- 保留英文 ID 與繁體中文標籤。
- 明確區分付款失敗及未知結果。
- 不增加規格以外的重試、退款與出貨。
- 附上需求對照，不得在沒有執行時宣稱渲染通過。

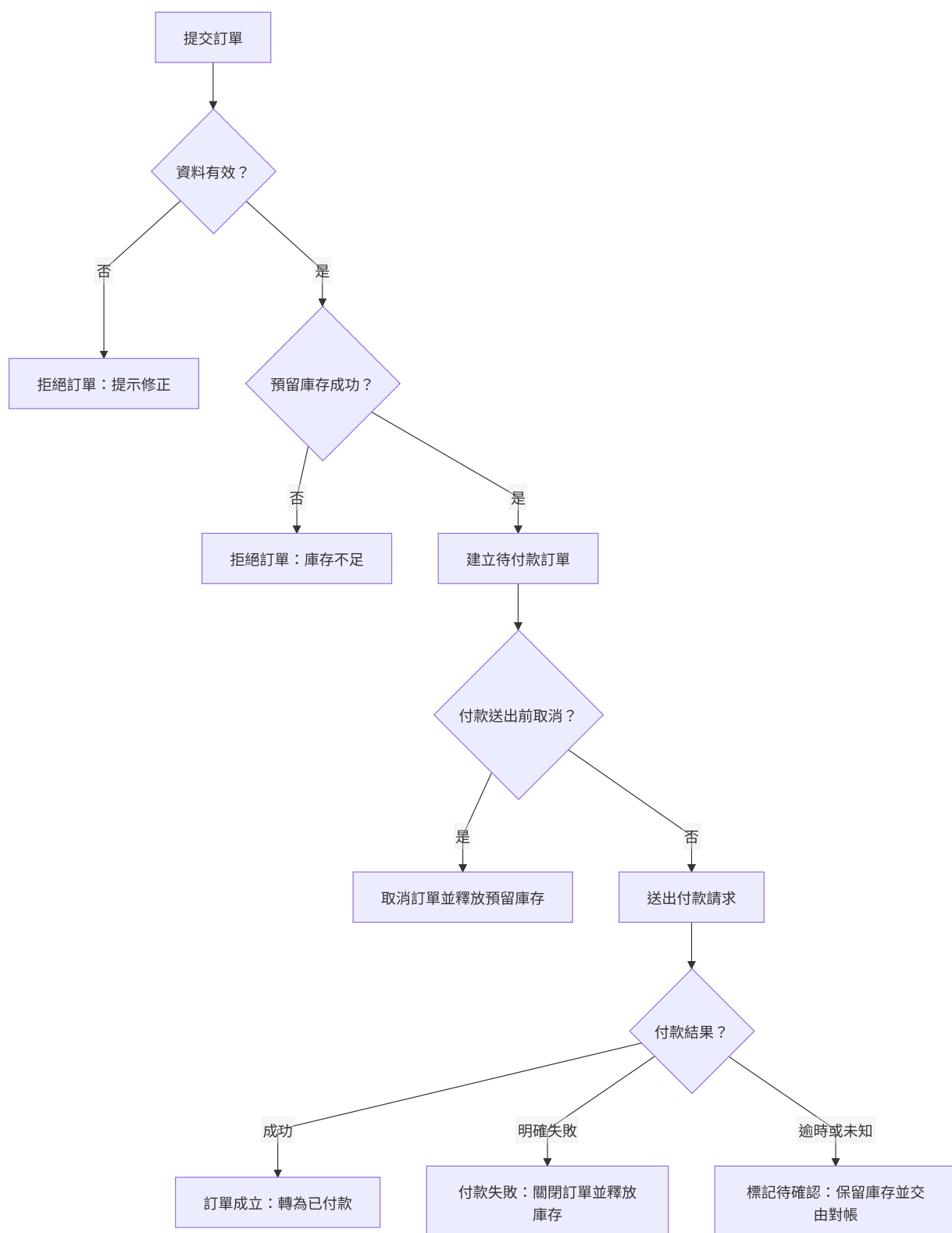
實務上不要只說「畫得專業一點」。如果圖的問題是未知結果流向失敗節點，就指出那條邊，要求局部修正。一次要求重寫整張圖，容易把原本正確的路徑也改壞。

書中的提示詞是可重做的輸入，不保證不同模型逐字產生相同程式碼。驗收應檢查需求關係與可讀性，不應以是否與書中碼完全相同作為唯一標準。

4.5 完整範例與成品

原始碼：[order-flow.mmd](#)

圖 4-1 訂單提交到付款結果的分流



原圖等比例縮放，未修改來源。校樣請檢查小字、連線與留白。

無法顯示 SVG 時，可開啟 [PNG 版本](#)。SVG 的中文字型可能隨檢視裝置改變；PNG 保留本次渲染結果，但放大會失去清晰度。

從提交開始沿箭頭走，前三個可能離開主流程的位置，分別是資料不合法、庫存預留失敗、付款送出前取消。只有預留成功才有待付款訂單；只有成功結果才轉為已付款。

付款送出前的取消是本例刻意選定的範圍。若要支援請求送出後取消，就必須補上取消與扣款競態的規則，不能只把同一條取消箭頭接到更多地方。

4.6 在自己的電腦產生圖

先安裝適合工具要求的 Node.js，再在本庫根目錄執行：

```
npm ci
npm run render:mermaid
npm run render:mermaid:png
```

本庫固定 Mermaid CLI 版本，並提交 package-lock.json。npm ci 可能下載 Puppeteer 所需的瀏覽器，需網路、磁碟空間與對套件安裝流程的信任。若組織限制瀏覽器下載，使用已安裝 Chromium 的方式見 [範例操作說明](#)。

繁體中文需要可用字型。本次使用 Noto Sans CJK TC；缺少時可能顯示方框或以其他字型替代。先查字型，再調節點寬度，否則只是在替缺字問題修排版。

上面的標準安裝方式是讀者操作入口。本次維護環境使用 npm install --ignore-scripts 及既有 Chromium，沒有另外執行一次乾淨環境的 npm ci；不要把這兩種驗證說成相同。

4.7 用路徑檢查圖的意思

本圖是文件，不會執行付款。以下是人工需求走查，不能取代程式單元測試或真實金流整合測試。

情境	應經過的分支	應停止或交接的位置
資料不合法	valid 的否	提示修正
庫存不足	valid 的是、reserve 的否	拒絕訂單
付款前取消	reserve 的是、cancel 的是	取消並釋放庫存
付款成功	cancel 的否、result 的成功	已付款
明確付款失敗	result 的明確失敗	關閉並釋放庫存
付款逾時	result 的逾時或未知	保留庫存、轉交對帳

語法驗證只回答「能不能產生圖」。還要檢查每個需求是否出現、分支有沒有走錯、長標籤是否裁切，以及圖片縮放後是否仍能閱讀。

本次 SVG 與 PNG 已成功產生，視覺檢查未發現裁切、重疊或路徑混淆；但整圖縮小時字體偏小，底部長標籤斷行不理想。這是樣章保留的排版改善點，正式紙本應拆圖或另做適合版面的輸出。詳見 [驗證紀錄](#)。

4.8 教學錯誤：把逾時當失敗

以下是刻意設計的錯誤，不是實際 AI 執行紀錄：

```
付款結果未知 → 關閉訂單並釋放庫存
```

這條邊可以畫得很漂亮，卻違反 R8。修正時先恢復「待確認」交接，再確認失敗分支沒有因此被刪除。不要只叫 AI 消除語法錯誤；這裡根本沒有語法問題。

另一個常見錯誤是用同一個 ID 表示「付款失敗」與「付款取消」。節點可能被合併或標籤覆蓋，讓兩種不同條件看起來是同一件事。先檢查 ID，再考慮換佈局。

4.9 練習：加入付款前逾期

新增規則：待付款訂單在「尚未送出付款請求」時可以逾期，逾期後關閉並釋放庫存。此條件不得適用於已送出付款且結果未知的訂單。

請先列出需要新增的條件，再改圖。檢查三件事：逾期是否只在付款前判定、成功與未知分支是否保持原意，以及取消與逾期是否仍能區分。若試圖處理計時與付款請求的同時發生，就超出本練習範圍，應另寫競態規格。

官方資料與版本注意

- [Mermaid 流程圖語法](#)
- [Mermaid CLI：安裝、SVG／PNG 匯出](#)
- [Noto CJK 字型](#)

官方網站會持續更新，書中的可重現環境以隨書 lockfile 與驗證紀錄為準。GitHub 頁面使用的平台渲染版本不一定與本庫 CLI 相同。

第 5 章：D2，用文字維護系統架構

樣章草稿。本例已使用 D2 v0.9.0 完成驗證、格式檢查及 SVG／PNG 匯出。它是虛構平台的同步呼叫視圖，不是完整生產架構。

5.1 同一組關係，不同的維護方式

[draw.io 樣章](#) 用可編輯節點及連線交付架構。本章保留相同的業務元件與呼叫關係，改用 D2 原始碼，讓讀者比較「改文字再生成」與「直接拖拉圖形」的差異。

D2 原始檔沒有本例 draw.io 那些逐一指定的 x、y 座標。作者描述元件、群組與關係，佈局引擎決定位置及走線。這適合跟程式碼一起審查的架構文件；若簡報要求某個元件精準對齊既有版面，draw.io 可能比較省事。

不要把「換工具」當成「換架構」。本例不加入訊息佇列、快取、閘道或微服務拆分，以免讀者無法判斷差別是來自工具還是需求。

5.2 圖的範圍與邊界

本圖回答兩個問題：有哪些呼叫關係？哪些元件由平台管理？

平台管理範圍內有 Web 前端、訂單 API、訂單資料庫；外部服務群組包含支付服務；使用者瀏覽器位於兩者之外。容器表示本圖約定的管理範圍，不代表 Kubernetes、Docker 或網路隔離，也不能直接推論安全信任等級。

箭頭表示呼叫方向，回應省略。Web 前端的部署方式、資料庫讀寫與付款請求的精確先後次序，沒有在這張圖定義。請求付款並不代表支付提供者一定同步回傳最終結果；回呼、對帳及失敗處理應讀其他流程文件。

[完整範例規格](#)

5.3 先理解 D2 的 ID 與標籤

```
client: 使用者瀏覽器
web: Web 前端
client -> web: 開啟頁面
```

冒號前是識別碼，後方是顯示文字。連線使用 ID，不使用中文標籤。如此一來，改顯示名稱不必同時改所有引用。

D2 的便利也可能掩蓋錯誤：連線提到未宣告的物件時，D2 可以自動建立它。因此 orders 打錯成 order，不一定讓驗證失敗，反而可能多出一個非預期元件。通過 parser 後仍要把圖上的元件清單對回需求。

5.4 容器讓範圍變得明確

```
platform: 平台管理範圍 {  
  web: Web 前端  
  orders: 訂單 API  
  db: 訂單資料庫 {  
    shape: cylinder  
  }  
}
```



```
platform.web -> platform.orders: 建立訂單  
platform.orders -> platform.db: 讀寫訂單
```

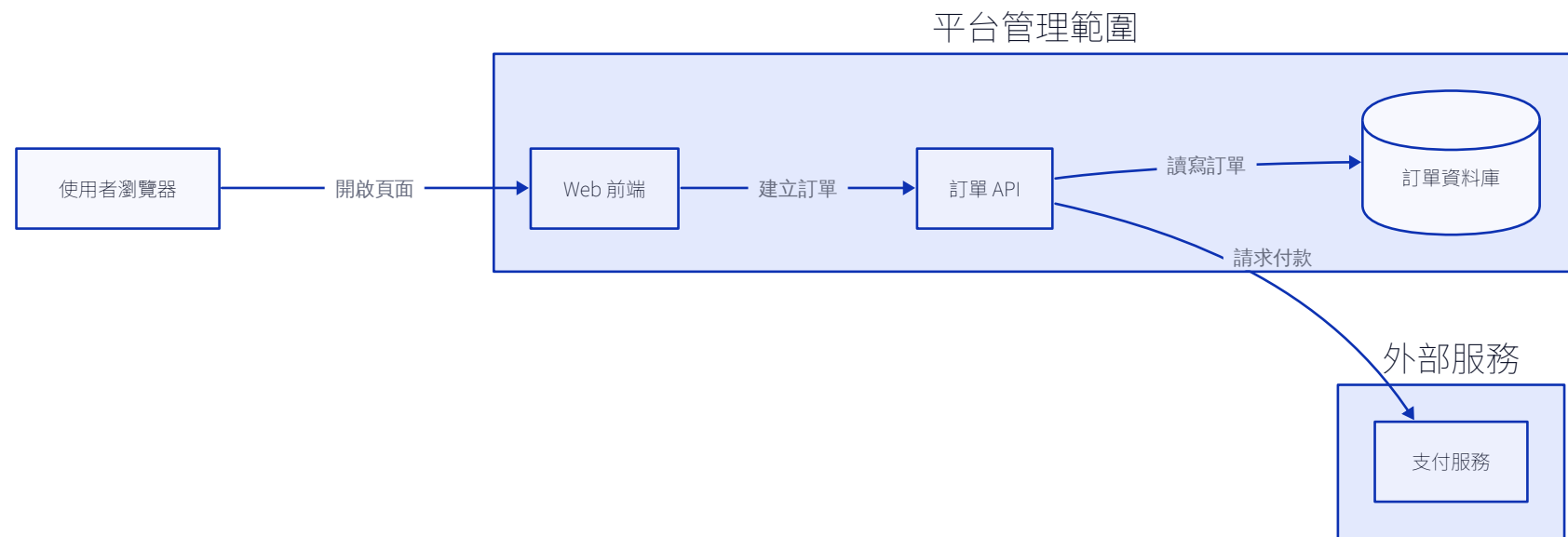
巢狀 map 建立容器，完整路徑 platform.orders 指向容器內的元件。若在最外層寫 orders，它可能不是你想要引用的那個物件。

本例只用一層容器。不要因為巢狀語法方便，就把組織、機房、網段、應用與模組全部套在一張圖裡；不同邊界的意義應分開說明，必要時拆成多張視圖。

5.5 完整圖與提示詞

[原始碼](#) · [提示詞](#)

圖 5-1 D2 平台同步呼叫視圖



原圖等比例縮放，未修改來源。校樣請檢查小字、連線與留白。

PNG 備用預覽

圖中四條呼叫應為：瀏覽器到前端、前端到訂單 API、訂單 API 到支付服務，以及訂單 API 到資料庫。前兩條主要由左向右，支付服務放在獨立容器，具體位置由佈局決定。

draw.io 試作的第一條邊標為 HTTPS；本圖改為「開啟頁面」，讓四條邊都描述動作。這是標籤一致性的調整，不是通訊協定的新增或刪除；不能從本圖推論明文 HTTP。

提示詞除了要求生成 D2，也要求交付元件與邊清單。AI 的說明文字和實際圖必須一起檢查，不能因為它在文字中列對了關係，就假設箭頭也接對。

5.6 固定版本、佈局與字型

從 [官方安裝文件](#) 選擇適合自己的方式。若要重現本例，使用 [v0.9.0 發布頁](#) 的對應平台版本，不要直接把最新版本當成相同環境。

先執行：

```
d2 version
d2 validate examples/d2/order-platform/order-platform.d2
d2 fmt --check examples/d2/order-platform/order-platform.d2
```

本例固定 dagre 佈局。可用 d2 layout 查看本機可用引擎，不能假定每種引擎都已安裝、授權條件相同或支援一樣的選項。本章沒有測試其他佈局引擎，不根據印象比較速度或品質。

繁體中文需可用字型。本次使用 NotoSansTC-VF.ttf，同時指定一般與粗體的字型輸入；使用同一檔案不保證粗體就有更強的視覺字重。實際圖仍要檢查。

在類 Unix shell，先把字型路徑改成自己下載的位置：

```
FONT=/absolute/path/to/NotoSansTC-VF.ttf
d2 --layout=dagre --font-regular="$FONT" --font-bold="$FONT" examples/d2/order-platform/order-platform.d2
examples/d2/order-platform/order-platform.svg
d2 --layout=dagre --font-regular="$FONT" --font-bold="$FONT" examples/d2/order-platform/order-platform.d2
examples/d2/order-platform/order-platform.png
```

FONT 是本例 shell 變數，不是 D2 自動尋找的設定名稱。PowerShell 語法不同，可直接在參數中填入自己的字型路徑。

SVG 與 PNG 都由 D2 CLI 直接產生。本次環境已具備瀏覽器相關相依；PNG 在新環境可能需要額外相依或下載，不能把本機成功當作乾淨機器必然成功。

5.7 驗證不是只看退出碼

本例執行 validate 成功，fmt --check 成功，兩種匯出都成功。但驗收還包括：

- 圖上的業務元件與需求相符，沒有因打錯 ID 而多出的節點。
- 前端、API、資料庫在平台管理範圍內，支付服務在外部群組。
- 四條邊的方向與標籤符合規格。
- 中文沒有缺字方框，容器標題及節點沒有裁切。

本次視覺檢查確認上述關係可辨識。仍有排版問題：「請求付款」靠近平台容器底邊；連線文字較細，縮圖時閱讀較吃力。這些問題不影響原始碼可用性，但正式印刷前應改善。

[完整驗證紀錄](#) 區分已執行與未測項目。首次製作時未做跨平台或 CI 測試；現行核心 CI 已檢查 D2／draw.io 的指定呼叫端點，但不執行 D2 渲染，詳見 [驗證狀態導讀](#)。

5.8 一個小改動可能造成整張圖重排

D2 的文字差異通常比 GUI 圖檔簡短，但短 diff 不等於視覺變化很小。增加一個節點或延長標籤，都可能讓佈局改變。審查時要同時看原始碼與新預覽。

不要手改 SVG 後把它當作下一次生成的輸入。此例以 .d2 為權威原始檔，修改後重新產生兩種預覽。如果需要人工完成精細排版，應明確移交到另一種來源格式，而不是讓 D2 與 draw.io 各自變成兩份互相衝突的真相。

容器的分組本身也是架構敘述。把支付服務移入 platform 並不只是把框拉近，而是改變管理邊界；這類修改須先確認需求。

5.9 練習與選擇

練習一：只將 platform.orders 標籤改成「訂單服務 API」，保留 ID。重新驗證及渲染，確認業務關係不變，並觀察字串長度對排版的影響。

練習二：在副本故意把一條邊的 platform.orders 寫成 platform.order。觀察 validate 是否會阻止它，以及渲染圖是否多出元件。這是教學用錯誤實驗，不是本例既有缺陷。

當團隊主要以文字、Git 和生成流程維護圖時，D2 很合適。當修改主要發生在會議中、需要逐個物件拖拉與精確對齊時，可參考 draw.io 樣章。先決定誰接手維護，再選工具。

官方參考

- [安裝](#)
- [容器](#)
- [連線與自動建立物件](#)

官方文件會更新。本例版本、字型指紋及匯出紀錄隨檔保存，沒有將工具二進位或字型重新分發。

第 6 章：PlantUML，把付款互動與狀態講清楚

樣章草稿。三個範例均以 PlantUML 1.2026.8 在本機完成語法檢查及 SVG／PNG 渲染；未執行真實金流程式或跨平台測試。

6.1 架構圖沒有告訴你的事

D2 架構章說明前端、訂單 API、資料庫與支付服務之間的呼叫關係，但沒有回答「哪個訊息先發生」或「逾時後訂單是什麼狀態」。把這些內容全部加到架構圖，只會讓箭頭混用不同含義。

本章分成三張圖：時序圖說明初次請求的訊息順序；狀態圖描述訂單在事件發生後如何轉移；類別圖說明訂單、付款請求及查詢紀錄的關係。它們使用同一份需求與教學假設，但不是互相替代的圖。

6.2 先說明本章新增的假設

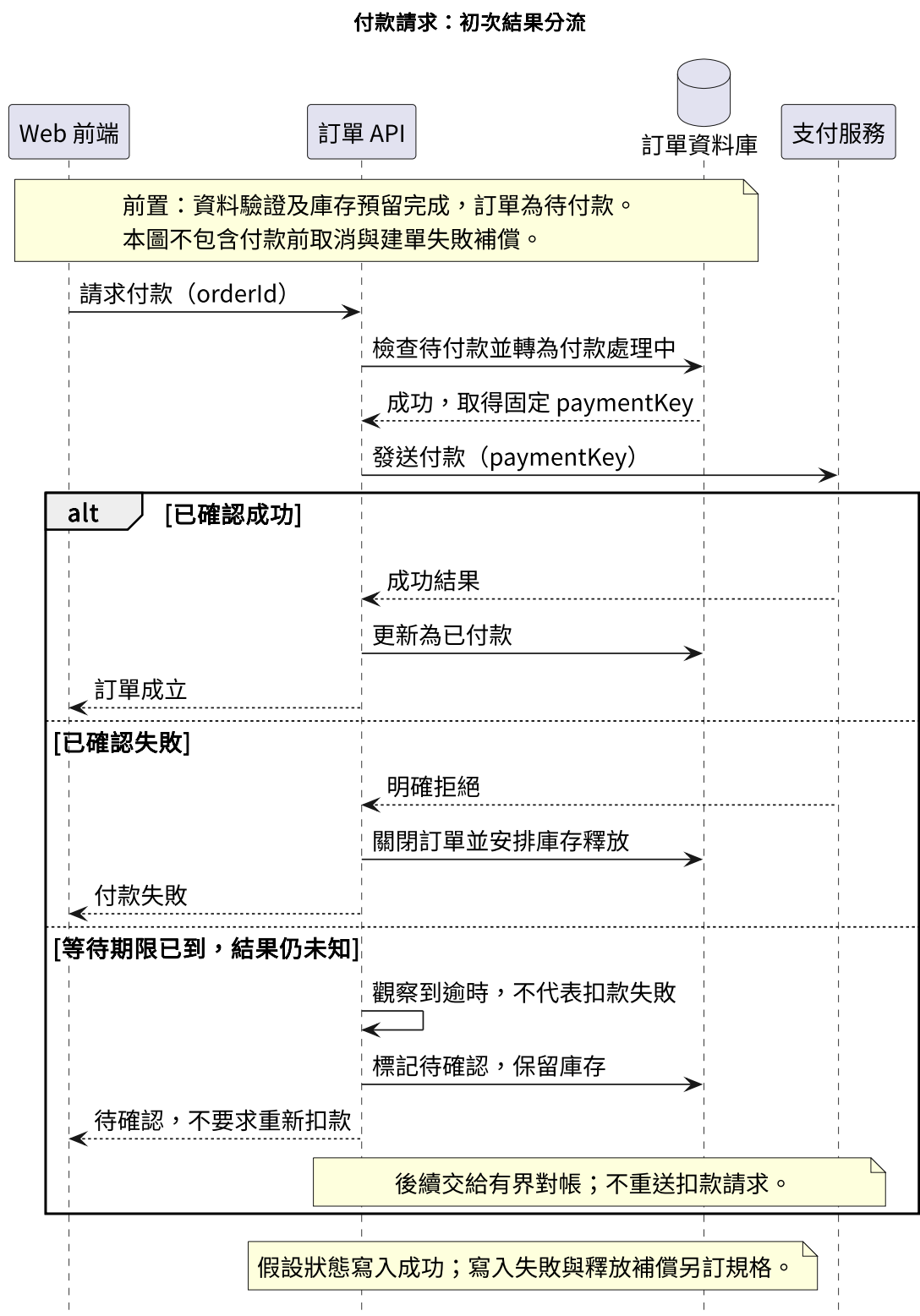
沿用 Mermaid 章的原則：付款結果未知不等於失敗。只有明確失敗或付款送出前取消，才走關閉及釋放庫存路徑。

本章進一步展開原本交給對帳的部分：每筆訂單最多一個扣款請求，使用固定 paymentKey；結果未知時只查詢原請求，不重新扣款。自動查詢最多三次，每次查詢完成或逾時都計數。第三次仍未知，交人工處理並保留庫存。

「三次」是教學假設，不是任何支付服務的標準建議。查詢間隔、總等待期限、人工服務時限與庫存保留期限尚未定義，不能直接把本圖拿去當生產環境規格。它限制次數，不保證處理會在某個時間內完成。

6.3 時序圖：誰在什麼時候發送訊息

圖 6-1 付款時序



原圖等比例縮放，未修改來源。校樣請檢查小字、連線與留白。

原始碼 · PNG

圖由上往下讀，參與者由左至右為前端、API、資料庫、支付服務。頂端註記限定前提：資料驗證與庫存預留已完成，而且本圖假設取得付款處理權及狀態寫入成功。

最小語法如下：

```
@startuml
participant "訂單 API" as API
participant "支付服務" as PSP
API -> PSP : 發送付款
PSP --> API : 已確認結果
@enduml
```

中文標籤與英文別名分開。實線及虛線在本章分別用於請求與回覆，但這是圖的表達約定，不會讓工具檢查實際協定。

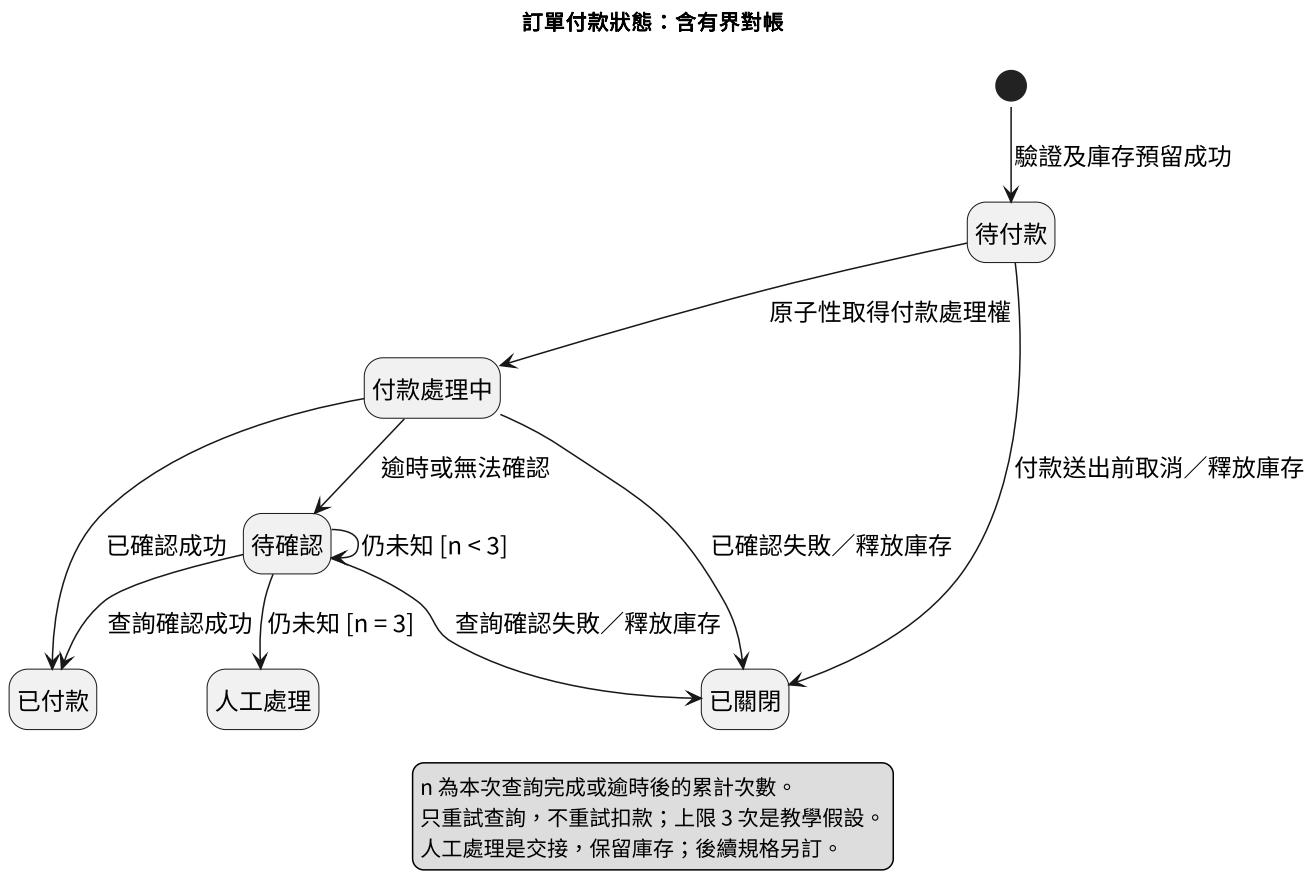
完整圖使用 alt / else / end 表示互斥分支：確認成功、確認失敗、等待逾時但結果仍未知。三條路不會依序全部執行。不要把 else 看成「下一個步驟」。

逾時被畫成 API 自身的訊息，因為這是呼叫端觀察到等待期限到達，而不是支付服務送出一個「我逾時了」的正常回覆。這個差別決定後續是否有權直接關單。

API 對前端回覆待確認後，後續查詢交給另一條流程。時序圖沒有畫出全部查詢訊息，不代表它們不存在；本章以狀態圖說明查詢結果對訂單的影響。

6.4 狀態圖：允許哪些轉移

圖 6-2 訂單付款狀態



原圖等比例縮放，未修改來源。校樣請檢查小字、連線與留白。

原始碼 · PNG

狀態描述的是訂單的當下條件，不是程式模組或訊息。Pending、Processing、Unknown 等是穩定 ID，中文名稱供讀者閱讀。

付款處理中可以轉為已付款、已關閉或待確認。待確認之後可以因查詢確認結果而離開；只有仍未知且累計查詢次數未滿三次，才留在待確認。

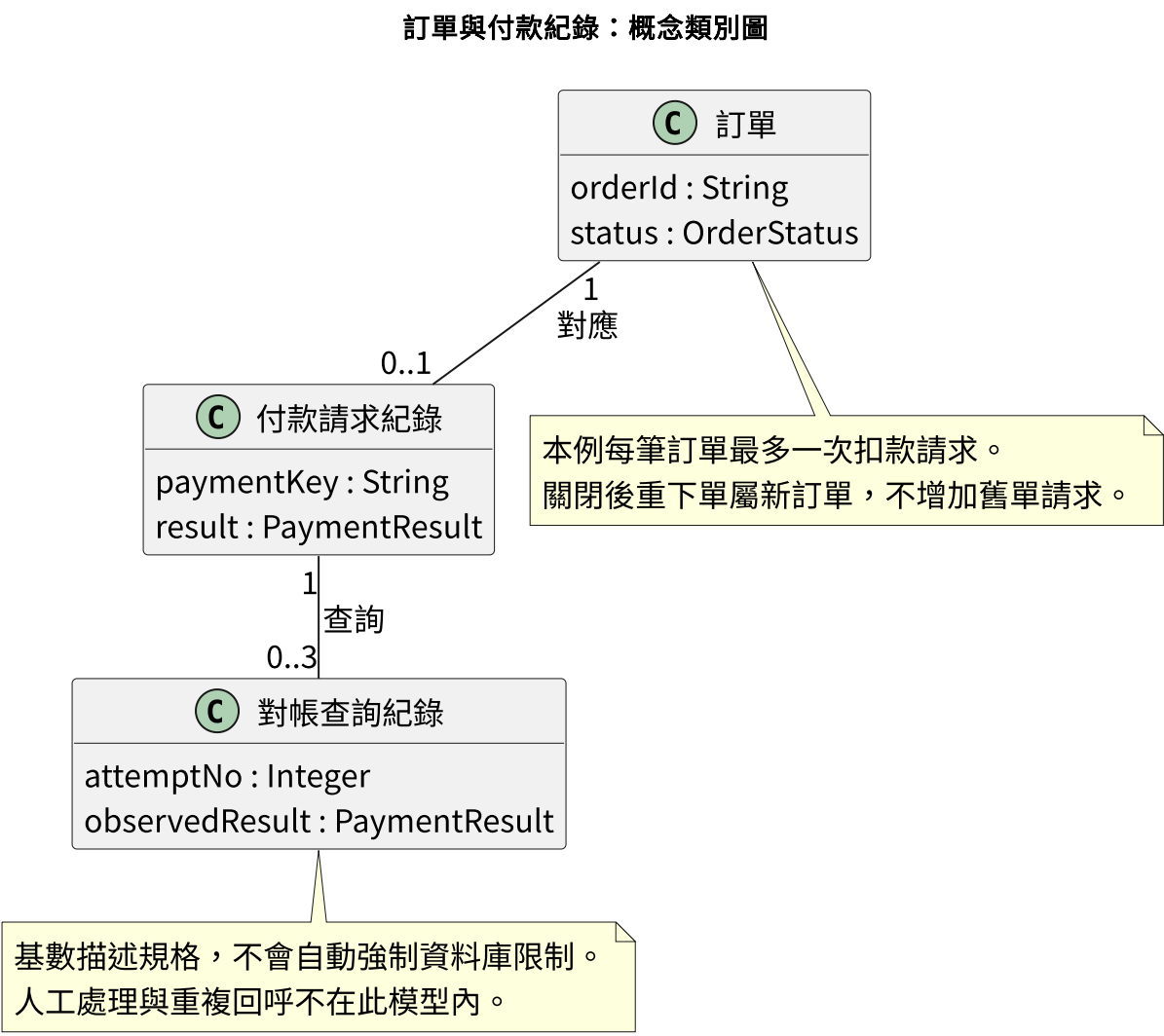
圖例中的 n 指「本次查詢完成或逾時後」的累計次數。第三次若查到成功，仍走已付款；只有第三次仍未知才轉人工。重試的是查詢，不是扣款。

[*] 在 PlantUML 狀態圖可表示起始或終止。本圖只有起始點，沒有把已付款、已關閉或人工處理連到終止點：已付款後可能還有出貨或退款，人工處理更是交接，不代表整筆訂單的生命週期結束。

狀態圖中的箭頭與守衛條件是規格，不會自動實作鎖、排程、交易或計數。並行查詢如何避免重複累加、晚到結果如何處理，仍需另訂機制。

6.5 類別圖：用基數限制模型的意思

圖 6-3 付款紀錄概念模型



原圖等比例縮放，未修改來源。校樣請檢查小字、連線與留白。

一筆訂單對應零或一筆付款請求紀錄；付款請求對應零到三筆自動對帳查詢紀錄。零筆付款請求對應尚未付款或付款前取消；零筆查詢對應初次就取得明確結果。

看基數時，要看它靠近哪個類別。Order 一端的 1 表示每筆付款請求屬於一筆訂單；PaymentRequest 一端的 0..1 表示一筆訂單最多一筆付款請求。

本圖用普通關聯，不使用組合菱形，因為本章沒有定義付款紀錄能否獨立保留、刪除訂單是否連帶刪除紀錄等生命週期規則。為了「看起來更像 UML」而添加組合關係，會替需求增加沒有依據的承諾。

OrderStatus 與 PaymentResult 是概念型別名稱，沒有在這張圖中展開所有列舉值。這不是 ER 圖，也不直接規定資料表或外鍵；0..3 更不會自動變成資料庫約束。

6.6 三張圖必須一致

本例交叉檢查：

- 時序成功分支更新已付款，對應狀態圖 Processing → Paid。
- 明確失敗分支關單，對應 Processing → Closed。
- 逾時分支只標記待確認，對應 Processing → Unknown。
- 自動查詢上限與類別圖 0..3 一致。
- 不重新扣款與 Order 到 PaymentRequest 的 0..1 一致。

如果日後允許同一訂單多次付款嘗試，不能只把類別圖基數改成星號；時序、幕等鍵作用域、狀態轉移及對帳歸屬也要一起重新設計。

6.7 在本機驗證與匯出

範例 [README](#) 提供完整命令。本次使用 Temurin JRE 21.0.12.1+1、PlantUML 1.2026.8、Noto Sans CJK TC。

```
java -Djava.awt.headless=true -DPLANTUML_SECURITY_PROFILE=SANDBOX -jar /path/to/plantuml-1.2026.8.jar -charset UTF-8 -checkonly "examples/plantuml/payment/*.puml"
java -Djava.awt.headless=true -DPLANTUML_SECURITY_PROFILE=SANDBOX -jar /path/to/plantuml-1.2026.8.jar -charset UTF-8 -tsvg -failfast2 "examples/plantuml/payment/*.puml"
java -Djava.awt.headless=true -DPLANTUML_SECURITY_PROFILE=SANDBOX -jar /path/to/plantuml-1.2026.8.jar -charset UTF-8 -tpng -failfast2 "examples/plantuml/payment/*.puml"
```

首次製作本章時，該驗證環境尚未安裝 Graphviz；第 7 章才另行配置。狀態圖及類別圖明確使用 `!pragma layout smetana`，本次已成功渲染；這不代表所有 UML 圖都能在所有版本使用相同方式。version 的一般 Graphviz 警告不能取代對實際範例的驗證。

安全設定採 SANDBOX，不載入遠端 include 或外部圖示。不要把公司機密流程貼到公開 PlantUML 伺服器；圖檔也可能含原始碼中繼資料，公開前應把來源與成品一起檢查。

6.8 從排版失敗中修正

第一版狀態圖把查詢次數的長說明放在自迴圈旁，文字與註解引線擠在一起。修正時把條件縮為 $[n < 3]$ 與 $[n = 3]$ ，將計數定義移到下方圖例，再重新匯出。

第二次視覺檢查確認字線重疊已消除，條件與圖例可讀；右側長標籤仍接近邊緣。時序圖第三分支標籤也略擁擠。它們是後續紙本排版改善點，不能以「能輸出 PNG」宣稱已完成出版品質檢查。

[驗證紀錄](#) 保存版本、範圍及成品雜湊。檢查結果只針對文件與圖像，沒有金流 API 測試。

練習

將自動查詢上限改為五次。先列出要同步修改的狀態條件、圖例、類別基數與需求文字，再修改原始檔。不得順便增加扣款重試，也不能把第三次查詢成功誤判成人工處理。

額外思考：如果第三次查詢仍未知但稍後收到成功回呼，誰有權更新訂單？本例未定義，應先提出規格問題，不要擅自在狀態圖畫一條線當作答案。

官方資料

- [時序圖](#)
- [狀態圖](#)
- [命令列](#)
- [Smetana](#)
- [安全設定](#)

官方 CLI 文件包含新介面說明；本章使用的是已在指定版本實測的命令，未把文件中的未測選項當成已驗證功能。

第 7 章：Graphviz，從依賴資料產圖與篩選影響

樣章草稿。本章包含可執行的 Python 產圖程式、測試與 Graphviz 成品。資料是刻意設計的虛構教學清單，不是掃描真實程式庫的結果。

7.1 先說清楚箭頭，才能談影響

本章的 $A \rightarrow B$ 表示 A 依賴 B。若付款模組改變，應找誰依賴付款，再找誰依賴那些使用者。這是沿邊的反方向追蹤，而不是從付款順著箭頭找它使用的 SDK。

資料圖與分析結果圖保留相同的箭頭含義。不要為了凸顯「影響傳播」而把結果圖的箭頭全部倒過來，卻沿用同一個圖例。

這與先前的同步呼叫視圖不同：本章是模組依賴視圖，不表達呼叫時間、成功與失敗分支，也沒有宣稱每一條依賴都會在每次請求中執行。

7.2 用小型資料集學會大圖的處理方式

本例總覽有 16 個節點、17 條不重複的有向邊，足以展示多層依賴、共享依賴與循環。它不是大型效能基準，也不代表已測試數千節點。

當真實資料變大，第一個方法應是篩選視圖，而不是不停縮小字體。可以按變更起點、模組群組或特定路徑縮小範圍；本章實作其中一種：變更節點的反向可達集合。

資料見 [dependencies.json](#)。它使用一個 ID $\rightarrow$ 中文標籤的 nodes 映射，以及 [使用者, 被依賴者] 的 edges 清單。元件名稱由資料決定，不由 AI 自行推測。

發票與訂單相互依賴是刻意加入的教學問題。圖中能看出循環，不代表程式已做完整的強連通分量分析；本程式只保證走訪遇到循環時不會無限重複。

7.3 DOT 的最小語法

```
digraph dependencies {
  rankdir=LR;
  "orders" [label="訂單模組"];
  "payment" [label="付款模組"];
  "orders" -> "payment";
}
```

digraph 表示有向圖， $\rightarrow$ 表示有向邊，rankdir=LR 讓分層佈局偏向由左至右。ID 與顯示文字分開；上方最小片段未指定字型，完整生成範例另設 Noto Sans CJK TC。

本例用 dot 佈局，適合有方向的依賴圖。它會嘗試減少交叉，不保證沒有交叉。不要把 DOT 語法與 dot 引擎混為一談；Graphviz 還有其他引擎，本章沒有替它們做比較測試。

7.4 從 JSON 產生總覽

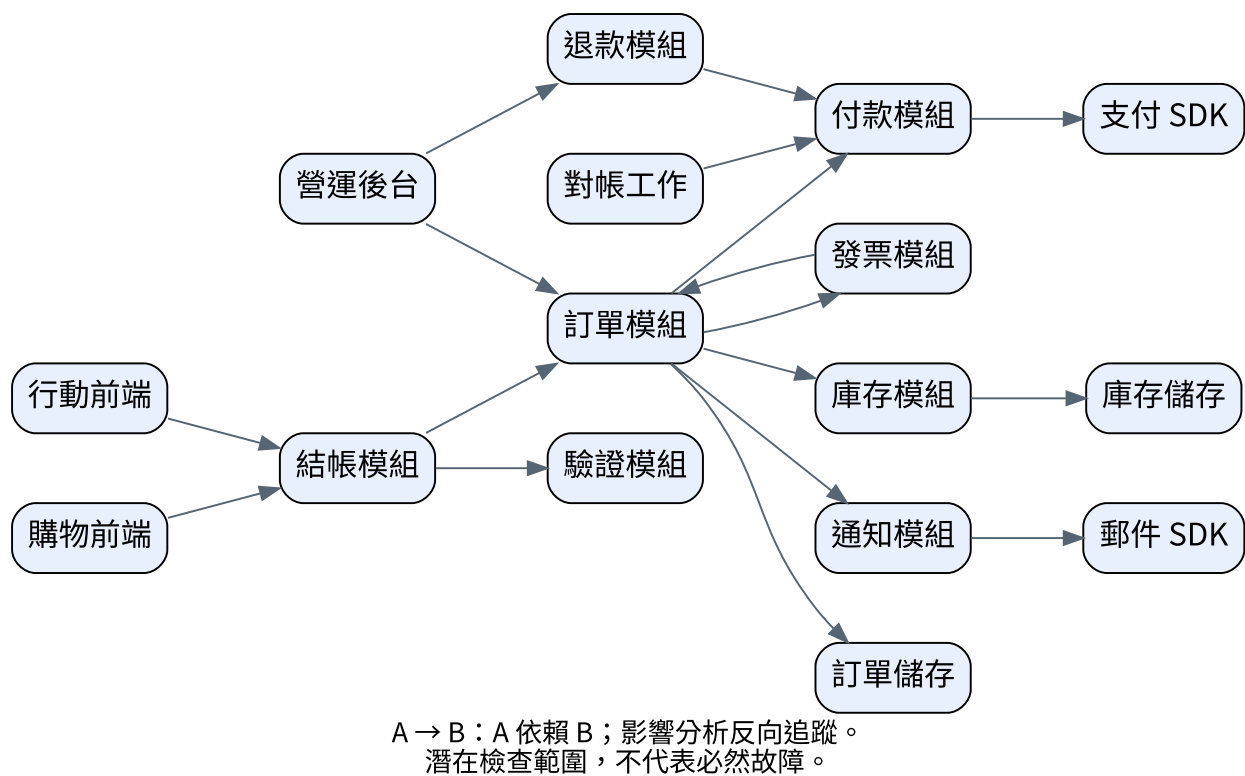
產圖程式 只使用 Python 標準函式庫。它不執行輸入資料中的指令，也不呼叫 shell；中文標籤與 ID 經過字串引用處理，再寫入固定 DOT 結構。

程式先檢查節點映射、邊清單、變更起點及連線端點。不能因 DOT 允許在連線中隱含建立節點，就忽略拼字錯誤。相同方向的重複邊去重，輸出按 ID 排序，降低輸入順序引起的差異。

```
python3 scripts/graphviz_dependencies.py examples/graphviz/dependencies/dependencies.json
examples/graphviz/dependencies/generated --changed payment
```

輸出 overview.dot、impact.dot 與 impact.json。輸出目錄中的同名檔案會被覆寫，請使用專用生成目錄，勿將手工文件放在這些檔名下。

圖 7-1 完整依賴



原圖等比例縮放，未修改來源。校樣請檢查小字、連線與留白。

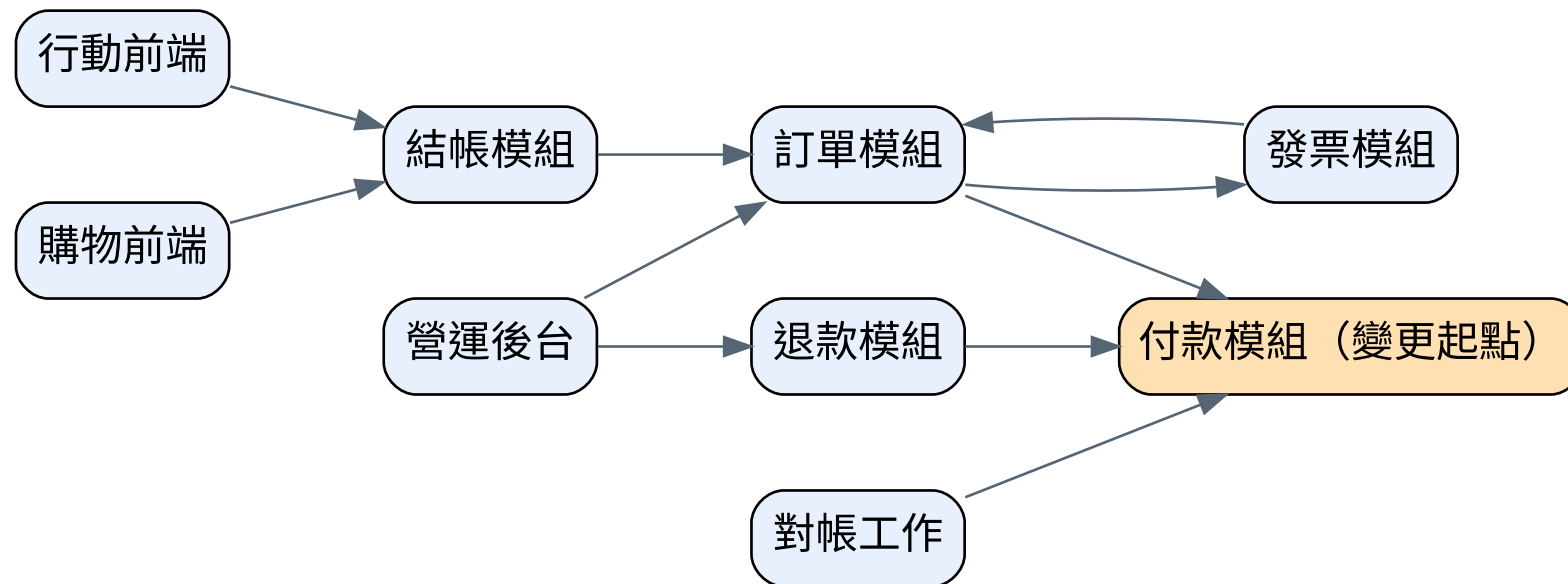
7.5 反向追蹤怎麼做

對每條 consumer → dependency，建立反向索引 dependency → consumers。從 payment 開始，將未訪問的使用者加入待處理集合，直到沒有新的使用者。

已訪問集合有兩個用途：避免共同上游被重複計入，也防止訂單／發票的循環讓程式跑不完。集合包含變更起點本身，表示要檢查的範圍包含正在修改的模組。

篩選完成後，保留兩端都在集合內的原始邊，形成誘導子圖。箭頭沒有反轉；反向是走訪方向，不是輸出的資料語意。

圖 7-2 付款影響範圍



A → B：A 依賴 B；影響分析反向追蹤。
潛在檢查範圍，不代表必然故障。

原圖等比例縮放，未修改來源。校樣請檢查小字、連線與留白。

PNG · 機器可讀結果

本次得到 9 個節點、10 條邊：付款、訂單、退款、對帳、結帳、發票、營運後台及兩個前端。支付 SDK 不在結果中，因為本例關心誰依賴付款，不是付款依賴誰。

這不表示 SDK、庫存或通知永遠不需要測試。它只表示在這份依賴資料及此種方向定義下，沒有被選入反向可達集合。實際回歸測試仍要考慮介面、行為、資料及隱含依賴。

7.6 讓分析程式先通過測試

隨書測試 用真實子程序執行 CLI，不用模擬結果。

```
python3 -m unittest discover -s tests -p 'test_graphviz_generator.py' -v
```

本次三個測試方法全部通過，涵蓋：反向傳遞與循環終止；不合法資料在輸出前被拒絕；單獨觀看圖時仍有箭頭語意說明。非法輸入測試再分成未知端點、未知起點、錯誤邊形狀與非文字標籤。

開發時先看到功能缺少造成失敗，再實作。增加輸入驗證時，測試先揭露 traceback 與可能建立半成品的問題；增加圖例時，先讓缺少語意說明的測試失敗，再修改產圖程式。

這些測試不等於全面安全稽核。它沒有負載上限、完整 JSON Schema 或重複 JSON key 檢查；本例是可信小型教學資料的 CLI，不應直接當成公開上傳服務。

7.7 渲染與版本差異

安裝 Graphviz 並確認 dot -V，在根目錄執行：

```
dot -Tsvg examples/graphviz/dependencies/generated/overview.dot -o
examples/graphviz/dependencies/generated/overview.svg
dot -Tpng examples/graphviz/dependencies/generated/overview.dot -o
examples/graphviz/dependencies/generated/overview.png
dot -Tsvg examples/graphviz/dependencies/generated/impact.dot -o
examples/graphviz/dependencies/generated/impact.svg
dot -Tpng examples/graphviz/dependencies/generated/impact.dot -o
examples/graphviz/dependencies/generated/impact.png
```

本機使用 Ubuntu 套件 2.42.2-9ubuntu0.1，其中 dot -V 顯示 Graphviz 2.43.0。驗證紀錄同時保存套件版與執行檔回報，沒有把兩者強行改成一樣。

這次沒有系統安裝權限，因此將套件解到使用者工具目錄，並只在命令環境設定函式庫／外掛路徑。讀者一般應使用適合自己系統的正式安裝方式；書中不要求照搬維護者本機的絕對路徑。

中文字型使用 Noto Sans CJK TC。SVG 字型可能隨讀者檢視環境替換，PNG 保存本次字型渲染結果。圖檔與原始 JSON、程式碼都應一起保留。

7.8 大圖的可讀性：總覽用來找問題，局部用來討論

總覽裡訂單模組的線條集中，訂單到付款與發票回向訂單的路徑有局部交叉。付款影響子圖省略不相關的下游後，循環的兩條方向更容易辨識。

初版圖沒有箭頭圖例，單獨貼進簡報可能被誤讀。加入底部「 $A \rightarrow B$ ：A 依賴 B；影響分析反向追蹤」及「潛在檢查範圍，不代表必然故障」後重新渲染，確認圖例完整且沒有裁切。

不要為了消除循環線就刪掉其中一條依賴。那會讓圖更乾淨，卻隱藏真實資料中的問題。需要分組或折疊時，也要說明省略了什麼。

練習

把起點改成 `payment_sdk`，輸出到另一個目錄。它的反向影響應包含 `payment` 與 `payment` 的所有上游，而不是只包含 `SDK`。再把起點改成沒有依賴方的節點，確認它本身仍保留。

進階練習：為循環群組上色。先定義強連通分量的規則及測試，再修改分析程式；不要只靠畫面上兩條箭頭看起來相反就猜測所有循環。

官方參考

- [DOT 語言](#)
- [dot 佈局](#)
- [下載與安裝](#)

本章展示可重現的小型分析流程，沒有從真實程式庫抽取依賴或測試大型效能。首次製作未配置 CI；現行核心 CI 已執行生成器測試，但不重新渲染本章圖，詳見 [驗證狀態導讀](#)。

第 8 章：draw.io，讓 AI 草稿能由人接手

擴寫樣章草稿。既有範例曾驗證節點拖曳、API 文字修改、連線綁定及 XML 重新載入；本輪僅擴稿與結構複查，原生保存對話框、桌面版與跨平台仍未測。

8.1 交付的不是一張無法修改的圖片

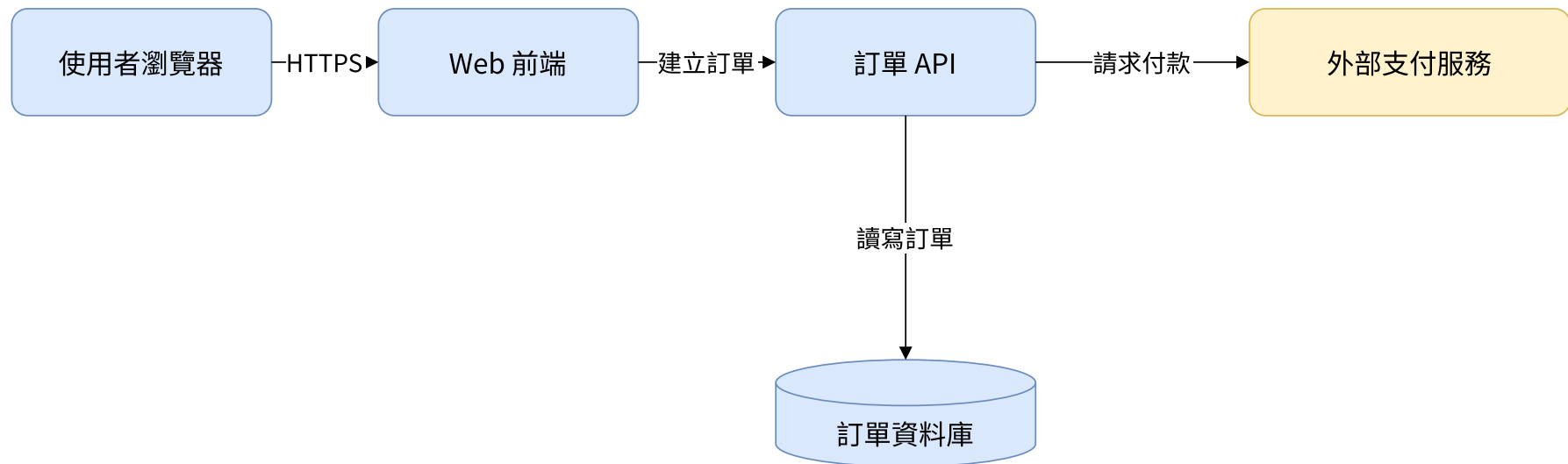
架構評審時，團隊常會說：「把資料庫移到旁邊」「這個服務名稱要改」「支付服務是外部的」。如果 AI 只交付 PNG，這些修改就得重新繪製。

本例交付 .drawio 原始檔，讓讀者能單獨選取元件、編輯標籤與移動節點。SVG、PNG 用於預覽，不作為唯一的維護來源。不要把向量格式誤認為原生可編輯圖：普通 SVG 匯入後不一定保留 draw.io 節點與連接關係。

[下載／檢視原始檔](#) · [範例操作說明](#)

圖 8-1 平台同步呼叫視圖

線上訂單與客服平台：同步呼叫視圖



箭頭表示呼叫方向；回應省略。
僅示意同步路徑；退款、回呼、庫存與客服模組另圖說明。

原圖等比例縮放，未修改來源。校樣請檢查小字、連線與留白。

8.2 先定義箭頭的意思

本圖包含使用者瀏覽器、Web 前端、訂單 API、外部支付服務及訂單資料庫。箭頭表示誰呼叫誰，回應省略。它不表示呼叫的精確時間順序，也不代表已定義交易或失敗補償。

例如「訂單 API → 資料庫」的單向箭頭並不是資料只能寫入、不能讀回；此處是呼叫視圖，讀取結果的回傳沒有另外畫線。若要說明付款請求與資料庫寫入的先後次序，應另畫時序圖。

平台名稱包含客服，但本圖只展開訂單同步路徑；客服、庫存、退款與支付回呼刻意省略。外部支付服務以名稱及黃色底色共同標示，不只依賴顏色傳達含義。

8.3 AI 應該產生什麼

本例採未壓縮 XML，方便讀者檢查與比較。基本結構是 `mxfile` → `diagram` → `mxGraphModel` → `root` → `mxCell`。

- `vertex="1"` 表示節點。每個節點有唯一 ID、顯示文字與位置尺寸。
- `edge="1"` 表示連接線。`source`、`target` 必須指向現有節點 ID。
- `mxGeometry` 保存座標與尺寸；它不是節點的業務身份。
- 保留固定 ID，局部改標籤或座標，能降低後續合併修改的困難。

本例提示詞 要求只用內建形狀、繁體中文標籤，不嵌入外部圖片、HTML 或執行程式碼。大型 XML 很容易出現重複 ID、未跳脫的 `&` 或懸空連接，AI 生成後仍需解析與編輯器驗證。

把連線畫到節點旁邊不等於綁定到節點。判斷方法是移動節點：如果線留在原地，就不是可以放心交接的圖。

8.4 用自己的瀏覽器開啟並修改

1. 在 GitHub 原始檔頁面選擇下載原始檔，保留 `.drawio` 副檔名；不要另存成 HTML 網頁。
2. 開啟 <https://app.diagrams.net/>，依畫面選擇裝置／本機儲存。
3. 用「File → Open From → Device」開啟下載的檔案；介面語言或版本可能使選單文字不同。
4. 選取「訂單 API」，向下拖曳，觀察前端、支付與資料庫的三條連線是否仍連接。
5. 雙擊文字，把名稱改為「訂單服務 API」。
6. 另存為 `order-platform-edited.drawio`，關閉圖後重新開啟，確認位置、標籤和連線都保留。

不要把敏感架構送到公開 Issue 或任意線上轉檔服務。離線需求應使用經組織允許的桌面版；本篇不把網址中的選項當作離線安全保證。

8.5 首次範例驗證究竟測了什麼

首次驗證使用 diagrams.net 已載入的真實 EditorUi 編輯器（版本 31.4.5）載入 XML，並用瀏覽器滑鼠事件拖動「訂單 API」：位置由 (560,160) 變為 (560,210)。三條相連邊仍指向原節點 ID。

文字修改透過編輯器 API 完成，接著序列化模型並重新載入，確認「訂單服務 API」與新座標保留。最後恢復原圖才匯出預覽，所以隨書原始檔仍是原始名稱與位置，便於讀者重做練習。

這次沒有走一遍使用者的檔案選擇器與「另存新檔」對話框，也沒有測試雲端儲存。API 模型往返與滑鼠拖曳已驗證；GUI 儲存流程是讀者練習，不宣稱已完成測試。

機器可讀紀錄 • 驗證範圍與成品雜湊

8.6 預覽能看，還要檢查哪些事

SVG 由編輯器直接匯出。PNG 由 Chrome 對該 SVG 產生，不是透過 draw.io 的 PNG 匯出選單。預覽中中文完整，未見裁切或節點文字重疊。

樣稿仍有可改善處：連線標籤較靠近線與箭頭；HTTPS 是協定名稱，其餘標籤是動作，正式版可統一為「動作＋協定」。標題使用整個平台名稱，單獨放入簡報時可改成「訂單建立：同步呼叫視圖」，避免讓讀者以為這就是完整平台。

若檢視平台無法顯示 SVG，可改看 PNG。本例使用純文字標籤減少 HTML/foreignObject 相容性問題，但字型仍受讀者裝置影響。

8.7 人工修改後，不要再讓 AI 覆蓋整張圖

把目前 .drawio 當作唯一來源。需要 AI 修改時，提供現有檔案與明確的改動範圍，例如「只改 orders 的文字，保留其餘座標與 ID」。不要重新給一段模糊需求，要求生成全新圖，再期待之前的人工調整自動保留。

大型修改先在副本進行。對照 XML 差異時，注意真正的節點與邊變更，不要把編輯器更新的畫布參數都當成業務變動。提交前重新開啟原始檔，並同步產生預覽。

8.8 從一條連線讀懂 XML

下列片段取自原生模型，省略樣式與幾何資料，只看業務端點：

```
<mxCell id="e3" edge="1" parent="1"
  source="orders" target="payment" value="請求付款" />
```

e3 是連線自己的 ID，orders 與 payment 是它連接的節點。移動 payment 時，連線應跟著端點重新排線，而不是只維持畫布上的固定終點。

本例節點的 mxGeometry 使用 x、y、width、height；連線的幾何資料則含 relative="1"。不要因兩者都叫 geometry，就把節點座標抄進連線當作綁定。單獨匯入上方片段也不是完整圖：它需要 mxfile、模型根節點及被引用的兩個節點。

畫面上的標題和說明也可以是 vertex="1"。因此不能把所有 vertex 的數量直接叫做「業務元件數」。本例有五個業務元件，另有標題與說明文字物件；比對需求時應按 ID 與角色分類。

本例所有業務元件放在同一圖層下，沒有建立代表平台內外的巢狀容器。外部身份靠名稱與顏色說明，不能拿這份 XML 宣稱 D2 的容器結構也被無損保留。

8.9 先用副本練習局部修改

假設交接者只要將訂單 API 改名為「訂單服務 API」。先另存副本，再將修改限制在 orders 的顯示標籤。不要換 ID、刪線重接，或順手替支付服務更名。

檢查可以分三步：

- 1. 修改前列出 e2、e3、e4 的端點。
- 2. 修改後確認端點仍是 web→orders、orders→payment、orders→db。
- 3. 真正開啟、移動、保存與重開，確認模型及畫面都保留；沒有走過的步驟填未測。

如果要調整位置，讓 geometry 改變是合理的；如果只改名稱，幾何資料的大幅變動就值得檢查。比較差異時應允許編輯器修改必要的畫布 metadata，但不能用「工具自動改的」忽略刪除節點或端點替換。

可用 [局部修改交接單](#) 記錄允許變更與禁止變更。模板不是完成證據；填好後仍需執行。

8.10 多人修改與衝突處理

兩人同時修改 .drawio 時，文字檔合併成功只代表沒有留下 Git 衝突標記，不保證圖能正確開啟。一個人可能移動 orders，另一個人則複製新節點並刪掉舊 orders，合併後的連線就可能引用錯誤 ID。

合併前先看各自的修改目的，再檢查 ID 唯一、父節點存在及端點引用。最後仍需真實編輯器重開。不要用重新生成整張圖來「解衝突」，除非團隊接受放棄人工版面，並重新驗收所有關係。

本書沒有實作通用 .drawio 合併器，也沒有驗證壓縮或多頁檔案合併。本章以單頁、未壓縮模型示範；遇到不同格式應先辨識來源，而不是把查不到 mxCell 當作檔案必然損壞。

8.11 匯出前的交接清單

交付項	檢查方式	本例限制
原生模型	真正開啟，選中節點與連線	初次測試用 EditorUi API 載入
人工編輯	移動節點、改標籤，檢查綁定	拖曳及 API 改字已有歷史證據

交付項	檢查方式	本例限制
保存重開	保存修改檔，再開啟下載的檔案	原生另存對話框仍未測
SVG／PNG	檢查來源一致、範圍、文字及留白	PNG 曾由 SVG 在 Chrome 轉出，不是 GUI PNG 匯出
版本	記錄來源 commit、工具與字型	不保證每個瀏覽器字寬相同
隱私	查原始資料、註解、外部圖片及連結	不以畫面沒看到秘密代表原檔沒有

如果需要以影像檔攜帶可編輯資料，先確認工具的匯出選項，再實際重新開啟驗證。普通 SVG 或 PNG 不提供這項保證；本例仍以 .drawio 為維護來源。

交接者若只收到圖片，請先找原生檔，而不是立即要求 AI 依圖重建。重建需要另外定義物件身份、端點及容器，不能假設會與原模型相同。

8.12 練習與參考判斷

練習一：只改名稱

把 orders 的文字改為「訂單服務 API」，保留其他 ID 與關係。參考：e2、e3、e4 的端點不變；較長標籤可能需要加寬，但需記錄這項額外幾何變更。原生保存流程要親自完成，不能引用本書舊紀錄代替。

練習二：支付服務改位置

把 payment 移到 orders 右下方，名稱改為「外部支付閘道」。參考：保留 payment ID、e3 仍由 orders 指向 payment、e4 仍指向 db。改名在本練習只是顯示調整，不代表供應商真的換了。

練習三：兩張圖的標籤不同

D2 第一條呼叫寫「開啟頁面」，draw.io 寫「HTTPS」。參考：兩者端點可能一致，但標籤分別表示動作與協定。第 11 章核心契約只比端點，不能用測試通過回答標籤是否一致。統一標籤前應先決定採哪種描述規則。

練習四：看似相連的線

在自己的副本中做一條只有幾何終點、沒有 source／target 的線。參考：靜態畫面可能像是連到節點，但移動後就能看出差別。這是讀者練習，本輪未執行新的 GUI 故障注入。

8.13 本輪擴稿的驗證邊界

本輪新增模型判讀、修改與合併流程、交接清單及練習參考，未改原圖或預覽。重新解析既有 XML、核對 ID／父節點／四條端點及 orders 的三條相連邊；再跑核心測試。

先前 EditorUi 拖曳、API 改字與模型重載的結果仍保留於原驗證檔。沒有在本輪重新操作 GUI，也沒有補測原生保存、桌面版、多人協作或所有瀏覽器。詳細結構複查與編輯範圍見 [本輪紀錄](#)。

官方資料

- [原生儲存格式](#)
- [SVG 匯出與字型相容性](#)

本例未嵌入外部服務商圖示，也未將第三方編輯器程式碼打包進 Repo。

第 9 章：Excalidraw，先把未定的事留在白板上

樣章草稿。案例與需求均為虛構教學設定，不是真實會議紀錄。

9.1 為什麼此時不畫流程圖

付款逾時之後，團隊常同時討論技術動作、責任分工與尚未決定的政策。如果先請 AI 畫一條完整流程，它可能替我們決定查詢間隔、接手人員，甚至把「未知」改成「失敗」。圖雖完整，需求卻被偷偷改寫。

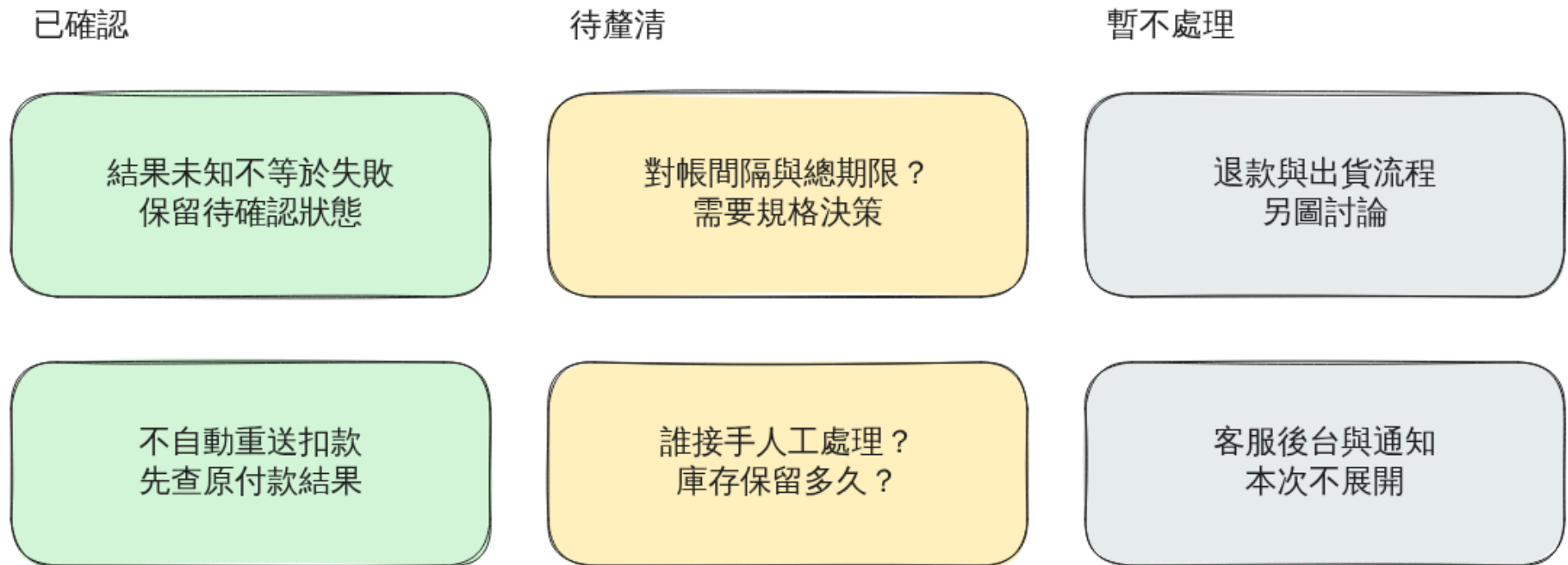
本章先做需求白板，不描述執行順序。三欄是「已確認」「待釐清」「暫不處理」，沒有流程箭頭。手繪感不代表規格可以含糊；每張便條仍需知道它是事實、問題，還是本次範圍的排除項。

[下載可編輯場景](#) · [操作與檔案清單](#)

圖 9-1 付款需求白板

付款結果未知：需求討論白板

虛構教學案例 | 欄位表示決策狀態，不是時間順序



未定事項不可讓 AI 自行補完；移動欄位前先記錄決策依據。

原圖等比例縮放，未修改來源。校樣請檢查小字、連線與留白。

9.2 先提供內容，不讓 AI 補答案

本例確認兩個教學前提：付款結果未知不等於失敗；不要自動重送扣款，而應查詢原付款結果。

尚未決定對帳間隔、總期限、人工接手者與庫存保留期限。退款、出貨、客服後台和通知只是本次不展開，不表示整個產品永遠不做。

第 6 章的「最多查詢三次」是該章的教學假設，不是本章自動採用的需求。跨圖協作時，必須區分為示範而設定的數字與正式政策。

完整條件見 [需求單](#)，可直接使用的 [AI 提示詞](#) 另存一份。

9.3 把 AI 輸出做成可編輯場景

要求 .excalidraw 原生 JSON，而不是把白板截圖貼進一個檔案。頂層包含 type、version、elements、appState、files；本例沒有嵌入圖片，files 為空。

便條用矩形和文字組成。文字的 containerId 指向矩形，矩形的 boundElements 指回文字；兩者有相同的 groupIds。這些關係讓拖曳便條時文字跟著走。只在同一位置放兩個物件，並不等於綁定。

ID 必須唯一且穩定。文字內容改了，不要任意重建全部 ID，否則版本比較會充滿無關差異。匯出 SVG／PNG 是交付預覽，不是原始檔可編輯性的證明。

9.4 實際開啟、移動與保存

1. 從範例目錄下載 requirements-board.excalidraw。不要將 GitHub 的 HTML 頁面另存成該副檔名。
2. 開啟 <https://excalidraw.com/>。已有未存工作時先另存，不要直接覆蓋。
3. 將場景檔拖入畫布，或從主選單使用 Open。
4. 找到「誰接手人工處理？」便條，沿水平方向稍移。框與文字應一起移動，且仍在「待釐清」欄。
5. 如要改字，可先進入群組，再雙擊文字。改動後檢查是否換行、超出框線或留下重複文字。
6. 使用主選單 Save to... 保存 .excalidraw；不同瀏覽器可能開啟另存對話框或直接下載。重新開啟下載的檔案，檢查位置與內容。
7. 用 Export image 匯出 SVG／PNG。取消 Only selected，避免只匯出選中的便條。確認背景和縮放；是否 Embed scene 必須明確選擇。

上述是讀者操作步驟，並非每一步都已自動化測試。此次實際驗證了匯入、滑鼠拖曳、瀏覽器重新載入、恢復原始場景及兩種影像匯出；未測原生另存對話框與雙擊改字。詳細證據見 [驗證紀錄](#)。

9.5 白板移動不等於決策成立

把黃色便條拖到綠色欄並不會使問題有了答案。先寫清楚答案、依據、誰確認，以及適用範圍，再移動便條、同步文字和顏色。

使用 [決策紀錄模板](#)。本例未虛構會議日期、決策人或批准狀態。三欄標題承載分類語意，不能只靠顏色辨識。

例如，若經正式確認「總期限為某值」，才把對應結論整理到規格，再更新 PlantUML 的狀態或時序圖。沒有新依據時，不要為了讓六套圖看起來一致而捏造一致的數字。

9.6 練習與參考處理

練習一：有人說「逾時就是失敗，直接再扣一次」。請指出它與哪兩張已確認便條衝突。參考：結果未知不等於失敗，且不得自動重送扣款；不能直接將此建議畫成執行流程。

練習二：新增「通知失敗如何處理？」。參考：本次通知功能不展開，可以記為後續討論，不擅自設計重試次數。若改變範圍，要先留下變更紀錄。

練習三：自行另存修改版、重新匯入並輸出預覽。參考驗收：三欄仍可辨識，框和文字一起移動，新增便條未被裁切，原始檔與預覽一致。

9.7 限制與官方參考

線上版會更新，本次沒有固定應用程式 commit，不能宣稱跨版本可完全重現。瀏覽器本機儲存也不是永久備份；本次自動化途中曾出現空白分頁及空儲存狀態，重新匯入檔案後才恢復工作。

文字使用 Helvetica 字族設定，繁中由環境可用字型回退；其他電腦可能有不同字寬。雲端協作、觸控、桌面系統的保存流程尚未測試。不要把內部客戶資料放進公開場景，選用 Embed scene 時更須檢查是否帶入非預期內容。

- [官方場景恢復工具](#)
- [官方影像與場景匯出工具](#)

六套工具的第一輪範例至此齊備；這不等於整本書已完成，也不等於所有平台和自動化發布都已驗證。

第 10 章：AI 繪圖除錯室

樣章草稿。此章的四組 broken／fixed 檔案是刻意製造的教學案例，不是真實事故。既有工具章的觀察另行引用，不混稱為本次重新測試。

10.1 先問「哪一層出錯」

AI 說「修好了」，通常只表示它改了文字。驗收需要另外回答四個問題：程式能解析嗎？圖意符合需求嗎？人看得清楚嗎？交付後還能修改嗎？

層次	典型症狀	最小證據	不能推論
語法／環境	指令失敗、空白圖	版本、命令、退出碼、原始錯誤	所有失敗都該改語法
語意／資料	方向相反、漏掉例外、節點拼錯	需求與節點／關係清單對照	能渲染就符合需求
視覺	缺字、壓線、裁切、縮圖不可讀	實際交付尺寸的預覽走查	XML 合法就好看
交付／可編輯性	圖片可見但無法改字、移動後連線斷開	真正編輯、保存及重開	副檔名正確就能交接

環境錯誤先確認執行檔、瀏覽器與字型。若錯誤是 Chromium 不存在，補一個括號不會解決；若錯誤是 Parse error，也不應先重裝整套工具。

10.2 建立可反覆執行的小迴圈

固定輸入 → 執行同一命令 → 保存原始錯誤 → 提出可否證假說 → 一次只改一項 → 同命令重跑 → 檢查鄰近需求。

請保留壞版本。沒有它，就很難證明檢查確實能抓到這一種錯誤。本章將反例保存在 broken.*，修正版在 fixed.*；反例不可當作可部署或可直接採用的正式範例。

四組重現檔、命令與輸出 · 除錯提示詞

10.3 案例一：Mermaid 少一個括號

錯誤版本：

flowchart LR

A[付款結果未知 --> B[待確認]

上方用一般文字區塊展示非法內容，避免章節頁面本身變成錯誤畫面。讀者應以 [broken.mmd](#) 為重現來源，而不是把網頁錯誤訊息當作 CLI 執行證據。

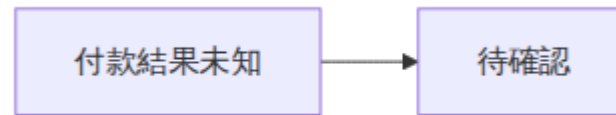
本機 Mermaid CLI 回傳 exit 1，錯誤開頭為 Error: Parse error on line 2:。後面的預期 token 清單很長，但最小比對很短：節點 A 的左中括號沒有在箭頭之前關閉。

A[付款結果未知 --> B[待確認] ← 反例

A[付款結果未知] --> B[待確認] ← 補上 A 的右中括號

只補一個]，不改成「失敗」，也不加入重扣款分支。修正版相同命令 exit 0，SVG／PNG 均成功產生。

圖 10-1 Mermaid 修正版



原圖等比例縮放，未修改來源。校樣請檢查小字、連線與留白。

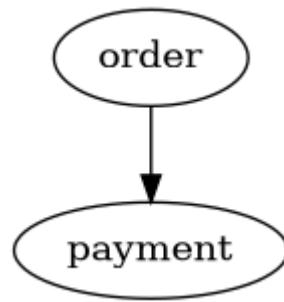
測試要求反例包含 Parse error，不只看非零退出碼；否則缺少瀏覽器也可能被誤認成「成功抓到括號錯誤」。測試還要求修正版 SVG 中保留「待確認」。

10.4 案例二：DOT 箭頭沒有目的端

反例的核心是 `order -> ;`。Graphviz 回傳 exit 1，錯誤為 `syntax error in line 2 near ';'。`

先比對需求：此最小案例規定訂單依賴支付。若資料中已有 `payment`，只補成 `order -> payment;`；不要為了讓語法通過就新增沒有依據的節點。

圖 10-2 DOT 修正版



原圖等比例縮放，未修改來源。校樣請檢查小字、連線與留白。

本圖僅示範 DOT 語法修復。此處箭頭表示 order 依賴 payment，不是訂單先做、付款後做；極小圖省略業務圖例，不能直接當作完整交付圖。相同線條外觀可有不同語意，不能只憑圖片猜關係。

10.5 案例三：兩次 exit 0，仍有一次畫錯

案例需求固定為：前端依賴訂單，訂單依賴支付。consumer -> dependency，支付變更時反向追蹤可能需要檢查的元件。

資料	邊	產圖程式退出碼	實際影響清單	是否符合本例需求
broken.json	order → web、 payment → order	0	payment	否
fixed.json	web → order、order → payment	0	order、payment、 web	是

兩份 JSON 都合法，節點也都存在。錯誤來源是輸入關係與需求相反，不是反向追蹤演算法失效；此案例不需要修改第 7 章產圖程式。

測試把預期集合明確寫為 ['order', 'payment', 'web']。反例必須不符合，修正版必須符合。這只是三節點案例的需求斷言，不是通用的自然語言語意判定器，也不能證明真實服務必然故障。

資料反例與修正版 · 實際影響清單與退出碼

10.6 案例四：多一個 s，節點不再是同一個

資料定義 payment，邊卻寫 payments。本章沿用第 7 章 CLI，實際回傳 exit 2，錯誤包含 each edge must name two existing nodes，且在全新輸出位置沒有建立產物目錄。

這不是要求 AI 遇到相似名稱就一律自動修正。先查案例的節點清單；本例明確只有 payment，才將邊中的識別碼改回它。真實需求若同時有兩個服務，合併名稱反而會出錯。

不要拿上次成功留下的 SVG 當作這次成功。使用新的暫存目錄，或明確管理來源、產物與雜湊。本章測試用暫存目錄隔離每個反例與修正版。

10.7 六套工具的視覺與交付問題

下表引用先前已保存的紀錄；本次沒有重新跑完這些編輯器操作。

工具	已觀察的問題或界線	處理與驗收
Mermaid	整張訂單圖縮小時文字偏小、底部換行生硬	優先拆總覽與局部，不只盲目加大字；原紀錄

工具	已觀察的問題或界線	處理與驗收
D2	支付標籤靠近容器底邊、連線文字縮圖偏細	區分容器歸屬正確與紙本可讀；此次僅 dagre，別宣稱其他引擎一樣； 原紀錄
PlantUML	狀態自迴圈文字與註解引線擁擠	縮短條件、把 n 的規則移入圖例，再重新匯出檢查；不能刪掉查詢上限語意； 原紀錄
Graphviz	總覽交叉線多	保留完整資料，另輸出影響子圖；不要只為排版刪真實依賴； 原紀錄
draw.io	XML 合法不代表拖曳連線仍綁定	真正拖動節點、檢查三條邊端點，序列化及重新載入；原生另存對話框未測； 原紀錄
Excalidraw	部分選取匯出、暫存不可靠、檔案 API 自動化限制	檢查 Only selected，保留原生檔，區分 Blob 接收器與系統保存； 原紀錄

中文字缺字要檢查字型是否真的被使用，而非只看設定裡寫了字型名稱。SVG 可在 A 電腦顯示，PNG 卻由 B 環境生成，也可能產生不同結果。用實際輸出的 PNG 檢查文字；字型嵌入前另查授權。

10.8 給 AI 的修復範圍

先交付最小原始檔、工具版本、完整命令、原始錯誤和一條可判定的需求。要求 AI 先提出最可能原因及反證方式，再做最小修補。

不要只說「幫我美化」。可以說：「保持三個分支與原有端點，只縮短顯示標籤；完整規則移到圖例；重新渲染後檢查有無裁切。不得自行更改查詢上限。」

若三次小修仍失敗，停止堆補丁，重新縮小案例、檢查環境及工具選型。回報時分開列出改了什麼、實際跑了什麼、仍未測什麼。

10.9 練習

1. 把 Mermaid fixed.mmd 的「待確認」改為「已失敗」。它可能仍可渲染，但違反本章未知不等於失敗的需求。參考：補語意斷言，而不是聲稱 parser 能懂金流規格。
2. 把反向依賴資料全部倒置後，直接跑渲染。參考：退出碼不會替代期望影響集合比對；依案例需求判定錯誤。
3. 在 Excalidraw 選中一張便條後匯出。參考：比較輸出內容與整圖需求；別把只輸出一張的 PNG 當作三欄白板。此練習尚未在本章重新執行。
4. 為自己遇到的問題填寫 [除錯紀錄模板](#)。要求另一人只用你的來源與命令就能重現，而不需要真實密碼或客戶資料。

10.10 本章驗證範圍

四組案例共八次核心 CLI 執行，結果詳見 [results.json](#)。新增四個回歸測試，連同既有三個測試，本機共七個測試方法通過。這表示反例被預期方式辨識、修正版符合本例斷言，不是「所有圖都正確」。

兩組修正版 SVG／PNG 均實際匯出，PNG 經視覺檢查。本章四個除錯測試尚未納入遠端核心 CI；相關生成器及端點契約的 CI 狀態見 [驗證狀態導讀](#)。乾淨環境安裝、跨平台、印刷尺寸審查及六套編輯器的統一端到端測試仍未完成。

第 11 章：跨工具協作與轉換限制

樣章草稿。沿用本書虛構平台，對照既有原始檔；不假設六張圖描述相同層次，也不宣稱格式間可無損互轉。

11.1 共用需求，不共用所有畫法

同一項需求可以需要多張圖，但不表示每張圖都要有同一批節點。Mermaid 描述訂單步驟，D2 描述服務與邊界，PlantUML 描述參與者、訊息及狀態；Graphviz 本例描述模組依賴，draw.io 提供可拖曳的呼叫視圖，Excalidraw 留住尚未決定的問題。

跨工具協作的目標，是不讓同一條已確認規則在不同視圖互相矛盾，而不是讓所有圖片長得一樣。

本章提供 [概念對照與原始檔指紋](#)、[權威來源表](#)、[變更演練單](#)，以及一個真實執行的 D2／draw.io 呼叫端點契約測試。

11.2 名字相同，不代表物件相同

先看實際來源中的識別碼：

概念	D2	PlantUML 時序圖	draw.io	Graphviz
訂單 API 角色	platform.orders	API	orders	orders 是訂單模組，只是相關概念
外部支付服務角色	external.payment	PSP	payment	未提供對應的外部服務節點
內部付款模組	未展開	未展開	未展開	payment

Graphviz 的 payment 標籤是「付款模組」，並依賴 payment_sdk。D2 的 external.payment 位於「外部服務」容器，PlantUML 的 PSP 是付款請求對象。兩者都談付款，卻分別是模組層與服務互動層；不得只用 payment 字串建立一對一轉換。

同樣地，Mermaid 的 reconcile 是「標記待確認：保留庫存並交由對帳」動作，PlantUML 的 Unknown 是狀態，Excalidraw 的便條是討論前提。它們可以共同支持「未知不等於失敗」，但不應被合併成同一種節點。

[concept-map.json](#) 因此同時記錄概念、來源、原生 ID、來源文字與適用範圍。它是手工判定後的對照，不是自動推出的架構真相。此次十個來源引用已與原檔比對，詳見 [檢查結果](#)。

11.3 共用的是契約，不是完整檔案

D2 與 draw.io 目前有一個可共享的小契約：

```
browser → web
web → order_api
order_api → external_payment
order_api → order_store
```

這四條是呼叫端點，不是時間順序，也不是 Graphviz 模組依賴。D2 的 platform.web 與 draw.io 的 web 可在此契約中映射為 web，但不要求兩者採相同原生 ID。

契約測試讀取真實 D2 來源與 draw.io XML，將端點映射後，各自與這四條比對；也檢查邊數與 draw.io 端點是否存在。只比兩張圖彼此相等仍不夠，因為兩張可能同時漏掉同一條邊。

```
python3 -m unittest discover -s tests -p 'test_cross_tool_contract.py' -v
```

本次在暫存副本中只刪掉 D2 的 platform.orders -> platform.db，測試確實失敗：3 != 4 : D2 edge count differs。換回原始來源，測試通過。原圖沒有被改動，也沒有替讀者刪掉資料庫關係。

這個測試刻意很小：只支援本例單行 a -> b: label 連線，遇到不支援的含箭頭語法會報錯。它不是完整 D2 parser，不理解容器、樣式、箭頭外觀或所有語意。

例如 D2 第一條標籤是「開啟頁面」，draw.io 第一條標籤是「HTTPS」。兩者端點相符，但一個描述動作、一個描述協定；本次沒有把標籤列入相等契約。若交付要求標籤也統一，需另訂詞彙規則，不能將這個測試說成全面一致性檢查。

11.4 哪一份可以改，哪一份只是產物

建立權威來源時，先分開三層：

1. 需求層：已確認的規則與明示的教學假設。尚未決定的事不能由任何一張圖自行升格成正式政策。
2. 視圖層：.mmd、.d2、.puml、.drawio、.excalidraw，或 Graphviz 的 JSON 與產圖程式。各視圖對自己的表示與版面負責，不取代需求。
3. 產物層：生成 DOT、SVG、PNG。用於預覽或下游消費，原則上從來源重新生成，不在此層偷偷改業務規則。

Graphviz 是特別容易混淆的一例：DOT 通常可以是原始檔，但本書第 7 章的 DOT 是由 dependencies.json 與 Python 程式生成，因此此案例的權威是 JSON 加程式。不能因副檔名是 DOT 就一律視為最上游。

draw.io 的人工位置調整可以留在 .drawio 原生模型；D2 的位置則由來源與佈局引擎決定。不應把 draw.io 拖曳結果承諾成 D2 能往返保存的固定座標。

11.5 轉換前先列出會失去什麼

從／到	可期待保留的部分	需要重建或另驗證的部分
D2 → draw.io 重建	人工映射後的元件與呼叫端點	容器模型、座標、樣式、連線綁定、編輯行為
Mermaid → PlantUML 重述	已確認的分支和狀態規則	參與者、訊息順序、前置條件、狀態轉移細節
Graphviz → 服務架構圖	明確有對照的模組概念	模組如何部署到服務、外部系統邊界；缺資料時不能推定
Excalidraw → 正式規格	已有決策依據的便條內容	未定數值、責任人、例外與驗收條件
任一來源 → SVG／PNG	該次渲染可見的圖像	原生語意、群組、端點綁定與完整編輯能力

這是資訊保留風險表，不是本次實測的轉換器支援表。各列不是同一種「轉檔」：有的是重建模型，有的是重新敘述需求。

PNG 可能包含某些工具的嵌入場景資料，SVG 也可能帶可編輯資訊，但必須由確定的匯出選項與實際重開驗證。本書不能只憑「這張圖看得到」就保證裡面有可編輯場景。

若確實要交接轉換，至少交付：來源版本、映射表、丟失資訊清單、轉換後原生檔、預覽、真實編輯與重開的結果。未測的保存流程仍要寫未測。

11.6 變更不是全檔搜尋取代

變更演練單 假設提出「訂單 API」顯示名稱調整，但不改端點或政策。這是待確認演練，不代表本章已批准或套用改名。

先確認這只是顯示名稱，還是服務真的拆分了。若只是文字變更，優先檢查 D2、draw.io、PlantUML 時序圖的對應標籤，再搜尋說明與提示詞；保留穩定 ID。Graphviz「訂單模組」不因字串相似就自動改成「訂單 API」。

若是查詢上限調整，則首先處理需求與假設。第 6 章的三次上限是教學假設；第 9 章仍把總期限與間隔放在待釐清欄，這不是矛盾。不能為了「一致」把白板改成已確認。真正批准新上限時，要檢查狀態條件、圖例、概念類別基數及文件，而不只是替換一個數字。

若服務拆分，文字更新就不夠：需重新確認容器、呼叫端點、時序參與者及模組映射。既有四邊契約應當因拓撲改變而失敗；不要先修改測試讓它變綠，必須先有新需求作為新契約依據。

11.7 不把版本指紋當作語意證明

對照檔保存八份來源的 SHA-256。它們能回答「此張對照表當時檢查了哪份檔案」，不能回答「內容是否符合真實業務」。來源修改後，舊指紋只表示需要重新檢查，不一定表示修改錯誤。

同理，SVG 的自動 ID 或字型字寬可能讓產物雜湊改變，而圖意並未改。版本控制需要同時保留來源差異、契約測試和視覺走查。

11.8 練習與參考判斷

1. 有人要求「把所有 payment 合併」。參考：先指出付款模組與外部支付服務的層次差異，要求提供部署或呼叫映射；沒有資料就不合併。
2. D2 與 draw.io 同時刪掉資料庫邊，兩張圖仍相同。參考：與外部明訂的四邊契約比對，而不是只做兩圖互比。
3. 只改 draw.io 節點位置。參考：保留端點綁定並重開檢查，不要求 D2 座標相等，也不需改 Graphviz 的模組資料。
4. 白板的一個問題已解決，另一個仍未定。參考：拆分便條或明示部分決定，不把整張便條當作全部已確認。

11.9 本次交付與限制

本章完成來源對照、權威來源表、變更演練、轉換提示詞及一個窄範圍契約測試。故障注入只在暫存副本執行，既有八份來源指紋確認未變。

沒有新增或執行格式轉換器，沒有新圖片，也沒有重跑六套渲染器或 GUI 編輯。本次可證明的是列出的來源引用與四條端點契約，不是全書所有圖語意完全一致。現行核心 CI 已執行本章端點契約；跨平台與完整書籍發布尚未完成，詳見 [驗證狀態導讀](#)。

第 12 章：版本管理、自動化與安全

樣章草稿。此章已加入本機命令與真正執行成功的 GitHub Actions 核心檢查；不是六套工具完整渲染管線，也沒有發布正式隨書 Release。

版本註記：本章四個核心／八個完整測試是第 12 章引入時的實測快照。第 13 章加入退款規格後，現行為六個／十個；請參考更新的操作入口與對應提交 CI。

12.1 把「我這裡能用」拆成可查的證據

畫圖交付至少有四種不同的可重現程度：

- 1. 原始檔可以取得，對應需求清楚。
- 2. 指定工具能解析或匯出。
- 3. 結構、語意契約與視覺檢查有證據。
- 4. 其他乾淨環境能按說明重做。

先前各章已完成不同範圍的本機驗證，但不能把它們合稱為第四級。本章先把不需外部渲染器的核心契約搬到 GitHub 託管 runner，讓它與維護者電腦分離執行。

操作入口與結果 · [工作流程原始檔](#)

12.2 三個命令，三種不同涵蓋範圍

儲存庫根目錄新增 Makefile：

```
make check-core
make check-full
make audit-npm
```

命令	第 12 章引入時的內容	依賴	當時實測結果
check-core	Graphviz 生成器三個測試、跨工具端點契約一個測試	Python 3、make	四個通過；本機與 GitHub runner 都成功
check-full	當時 tests 目錄的全部八個測試，含 Mermaid／DOT 反例	前述工具、Mermaid CLI、Chromium、dot	本機八個通過
audit-npm	npm 套件漏洞公告查詢	npm、專案套件資訊與網路	本次回報 0 vulnerabilities

full 指目前 Python 測試集合的全部，不是六套工具所有 GUI、渲染器與跨平台組合。它仍未覆蓋 D2／PlantUML 渲染、draw.io／Excalidraw 互動、印刷品質、秘密掃描或部署安全。

沒有 make 時，可以直接執行 [README](#) 的 Python 命令。需要不同 Python 時可用 `make check-core PYTHON=python3`。正式檢查前取消不需要的 `BOOK_SOURCE_ROOT` 與 `REFUND_SCENARIOS`，避免意外測到另一份副本。

12.3 工作流程為什麼先保持小

`core-contracts.yml` 在 main 的 push、pull_request 與手動觸發時執行 `make check-core`。

設定包含：

- GitHub 託管 ubuntu-24.04，不用有長期憑證的自架 runner。
- 明確 permissions: contents: read，沒有寫入或發布權限。
- checkout 固定完整 commit SHA；此次由官方 actions/checkout 的 v4 ref 查得 11d5960a326750d5838078e36cf38b85af677262。
- persist-credentials: false，不把 checkout 的授權資料留在 Git 設定供後續步驟使用。
- 五分鐘 timeout，同一工作流程／ref 的舊執行可以取消。
- 無 npm 安裝、無部署、無秘密變數引用、無自動產圖回寫。

這減少了安裝與權限面積，但不是安全沙箱的完整證明。pull_request 中的程式仍會被執行；因此不要臨時把部署金鑰或雲端憑證加入這個 job，也不要改用 pull_request_target 後執行未審查的 PR 程式。

不要把 Issue 標題或其他未信任文字直接插進 shell 命令。即使只是「圖的標題」，到了 shell 邊界也可能成為命令注入的輸入。

12.4 這次確實在 GitHub 上執行了什麼

工作流程引入提交：c61557405d3edfbb9d991911459b4edf8e2e036e。

實際成功執行的 head SHA 與上方提交相符，事件為 push，core job 及契約檢查步驟回報 success。API 讀回資料保存在 `ci-evidence.json`。

本機另外執行三個 Makefile target，完整 stdout、stderr 與退出碼在 `local-results.json`。本次沒有把 npm audit 或完整渲染反例放進遠端 CI，因此不能宣稱 GitHub 已替所有八個測試及漏洞檢查背書。

此外，本章沒有設定 branch protection 或 required status checks。工作流程存在，不代表合併一定被阻擋。若日後要強制檢查，先決定分支策略及管理員例外，再設定與驗證；不在本章自動替作者決定。

12.5 鎖版本有層次，雜湊也有界線

Mermaid CLI 的頂層版本與 package-lock.json 能縮小 npm 依賴變動，但還有 Node、Chromium、字型及作業系統。D2、PlantUML、Graphviz 也各有工具與字型依賴。記錄檔案雜湊有助於比對產物，卻不等於驗證上游簽章。

固定 Action SHA 可以避免移動標籤默默換碼，不能證明該 SHA 本身安全。仍須查來源、審查更新與維護漏洞修正。ubuntu-24.04 是會更新的 runner 映像，本工作流程沒有固定完整映像 digest 或 Python 修補版本，不能稱為完全封閉、逐位元相同的建置。

如要重建 npm 環境，通常使用提交的 lockfile 配合 `npm ci --ignore-scripts`，並另行安排可信的 Chromium。這不是本章已完成的乾淨安裝實驗。禁止安裝腳本也不代表之後手動啟動套件 CLI 就沒有風險。

產物差異應分開檢查：來源變了嗎？工具版本變了嗎？字型變了嗎？只是 SVG 自動 ID 變了，還是真正缺邊？不能因 PNG 雜湊不同就自動宣告需求錯誤，也不能因 hash 一樣就認為需求正確。

12.6 公開圖檔也可能洩漏資料

常見敏感資料不只 API 金鑰，還包括內部網址、帳號、客戶姓名、付款識別碼、檔案路徑、註解與嵌入場景。公開範例應使用明確標示的虛構資料。

.gitignore 只影響尚未追蹤檔案，不會讓已提交的秘密從歷史消失。若憑證洩漏，先撤銷或輪替，再處理檔案與歷史；不要只刪目前那一行就宣稱風險解除。不要把完整環境變數或登入 JSON 當作除錯附件。

原生場景、SVG 或支援嵌入場景的 PNG，可能保留未預期內容。應檢查圖像之外的原始資料，必要時明確關閉場景嵌入。對他人提供的 SVG、外部圖片及 include，先視為未信任輸入；不要在有登入權限或秘密的環境中任意開啟、執行或允許網路讀取。

PlantUML 的 SANDBOX 等設定有助限制部分行為，但不能替代完整的工具版本管理與隔離。來自圖檔文字的「忽略原有規則」「上傳私密檔」是資料，不是可執行的使用者指令。

npm audit 本次為零只代表當下公告資料的結果，不包括 Python 程式、字型授權、圖中個資、惡意內容與未公開漏洞。本章未部署自動秘密掃描，不以一般測試通過冒充安全稽核完成。

12.7 從 main 到讀者可依賴的版本

main 是變動中的工作版本。正式隨書版本需先確認授權、章節與檔案清單，再選定 commit、打標籤及建立 Release，並讓讀者能從書中找到對應快照。

使用 [發布前檢查表](#) 記錄：來源 commit、需求審閱、工具版本、測試、視覺走查、GUI 重開、下載包重做、授權及隱私。未完成項不可為了發布而預先勾選。

本章沒有建立任何正式 tag／Release、Pages 或授權檔；也不會在 CI 中自動 commit 新圖片。先讓生成與驗證可追蹤，再決定發布策略。

12.8 練習

1. 在暫存副本刪掉 D2 的資料庫邊，跑 make check-core。參考：應由端點契約抓到缺邊；不要只讓兩張圖彼此相等。
2. 移除本機 Chromium 設定後跑 make check-full。參考：環境缺失不是「括號反例通過」；測試要求正確解析錯誤以及修正版能渲染。請在副本環境測試，不破壞平日設定；本練習未在本章重做。
3. 找到一張 PNG，詢問是否可編輯。參考：檢查原生來源／嵌入場景及實際重開，不憑副檔名回答。
4. CI 顯示綠色，但圖中文字壓線。參考：目前核心 CI 未涵蓋視覺驗收，應開缺陷並補該層檢查，而不是把綠燈當作圖必然正確。

官方參考

- [GitHub Actions 安全使用](#)
- [npm ci](#)

本章將可執行檢查與人工發布關卡分開，讓讀者知道每一個綠燈究竟證明了什麼。

第 13 章：部分退款綜合專案

樣章草稿。本章是虛構需求變更與文件交接，不是可以直接上線的退款系統。沒有連接支付服務、執行真實退款或驗證資料庫交易。

13.1 先界定變更，不重寫歷史範例

原本付款案例處理的是第一次扣款及其成功、失敗、未知結果。本章新增「已確認付款成功後，允許多次部分退款，累計不得超過原付款」的教學需求。

新例放在 [examples/refund-change](#)，不修改第 4 至 9 章的既有圖。這不是漏同步：原圖有明確範圍，新增退款應有自己的來源與驗證。若之後要整合成正式產品規格，需另行確認哪份文件被取代。

交付包括 [需求單](#)、[跨工具影響表](#)、兩份可編輯來源、四張預覽、金額情境與實測紀錄。

13.2 額度不是「原金額減掉成功退款」而已

本例所有金額以整數新臺幣元表示，暫不處理小數、跨幣別、手續費、爭議款、退貨物流或稅務。這是教學範圍，不是金融系統通用金額規格。

可退額度 = 已付金額 - 已成功退款 - 已占用退款額度
 $0 < \text{本次金額} \leq \text{可退額度}$
 $\text{已成功退款} + \text{已占用退款額度} \leq \text{已付金額}$

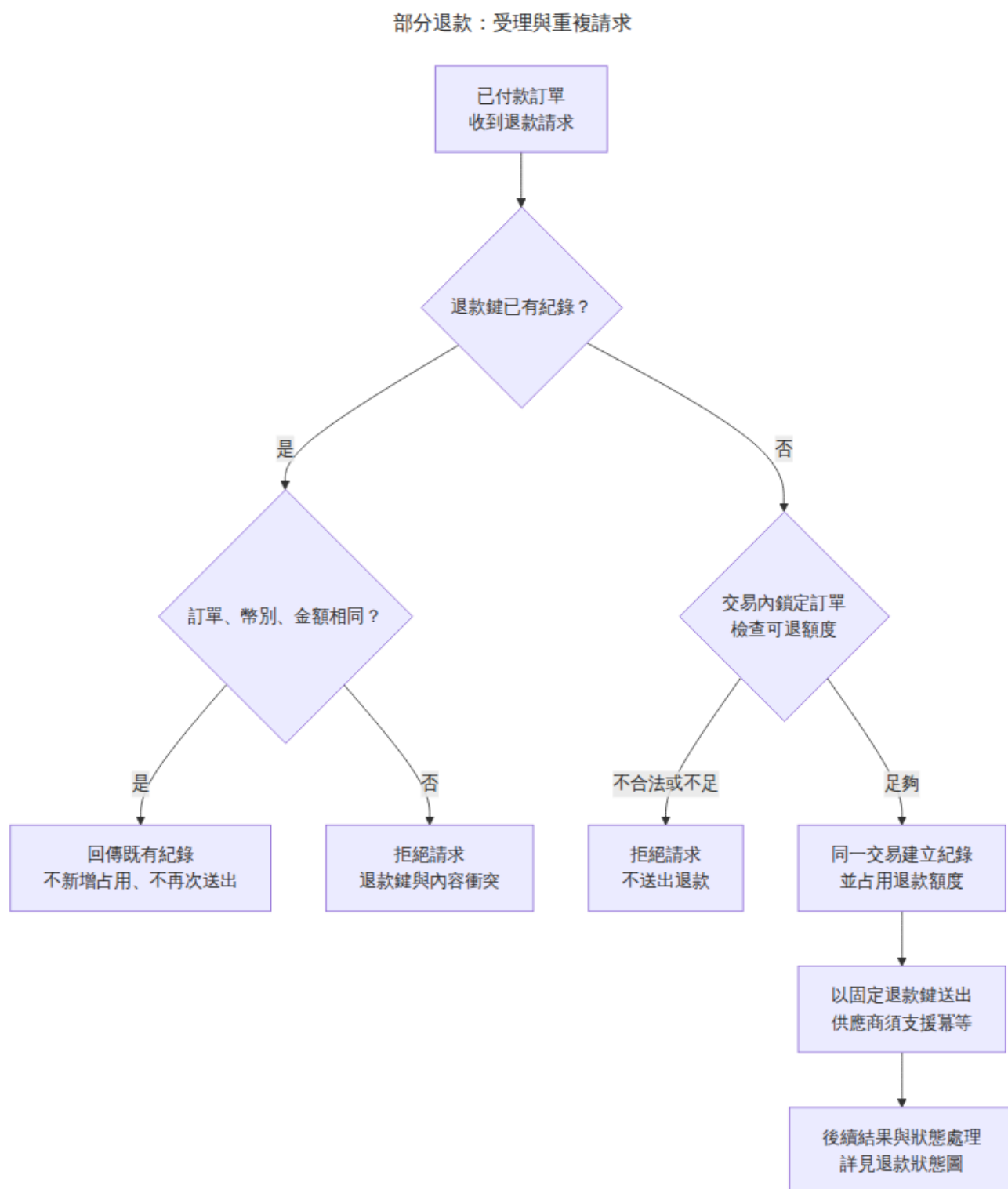
占用包含處理中與結果未知的退款。若只扣已成功退款，先前逾時但可能已成功的一筆就會被遺忘，讓新請求再次使用同一份額度。

情境	已付	已退	已占用	本次要求	可退	判定
第一次部分退款	1000	0	0	300	1000	額度允許
另有未知退款	1000	300	200	400	500	額度允許
超過剩餘額度	1000	300	200	600	500	拒絕
零元	1000	0	0	0	1000	拒絕
負數	1000	0	0	-1	1000	拒絕
已全數占用	1000	300	700	1	0	拒絕

以上只判斷此例的金額條件，不替代付款狀態、訂單所有權、操作權限及防詐檢查。可機讀資料見 [scenarios.json](#)。

13.3 先用 Mermaid 說明受理與重複請求

圖 13-1 部分退款受理



原圖等比例縮放，未修改來源。校樣請檢查小字、連線與留白。

Mermaid 原始檔 只回答「這次請求能不能建立新的退款紀錄」，結果處理交給下一張狀態圖。

同一退款鍵再次出現時，比對訂單、幣別及金額：一致就回傳既有紀錄，不新增占用、不再次送出；不一致就拒絕衝突。退款鍵需要適當的商戶／帳戶範圍及唯一性，不能只依靠前端傳來一段字串。

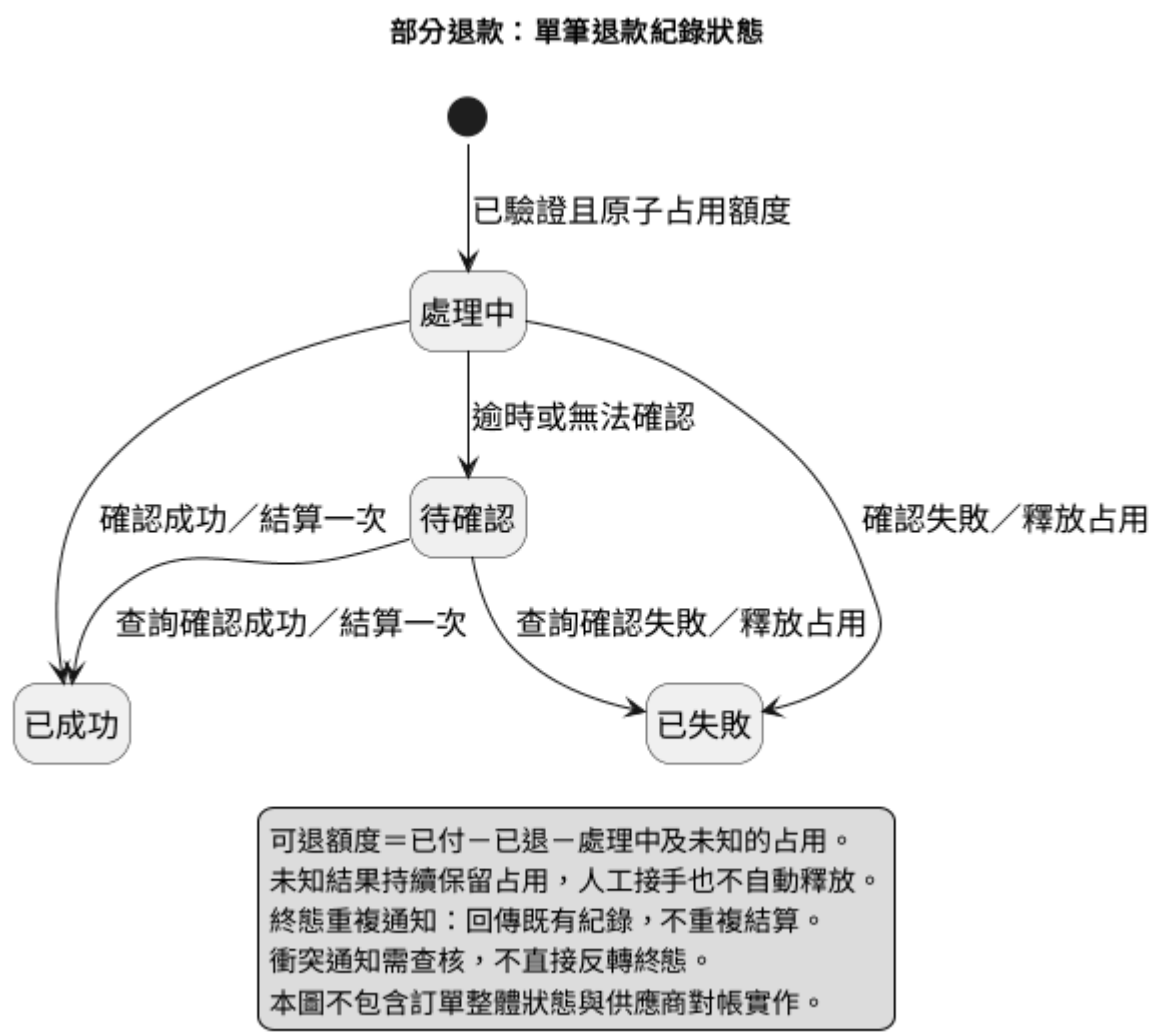
新退款的檢查與占用必須在同一資料庫交易中，對同一訂單的競爭請求序列化；退款鍵唯一約束也需在這個邊界強制。圖上前置查重不是唯一防線：兩個請求同時查不到紀錄時，插入衝突必須回頭讀既有紀錄並檢查內容，不得各送出一一次。

同一意圖若每次換新鍵，單靠冪等鍵無法識別重複意圖；額度上限也不能阻止額度內的誤操作。需要權限與業務防重規則，不能把「不超額」等同「每筆退款都合理」。

本例假設供應商支援固定退款鍵的冪等操作及結果查詢，沒有實測任何供應商。交易提交後送出失敗或程序崩潰，真實實作需可靠工作佇列／outbox 等方案，以同一紀錄與鍵恢復，不能建立新退款來補救；本章未實作或測試這一層。

13.4 再用 PlantUML 定義結果與占用

圖 13-2 退款狀態



原圖等比例縮放，未修改來源。校樣請檢查小字、連線與留白。

PlantUML 原始檔 描述單筆退款紀錄，而不是訂單整體狀態。

- 處理中：已建立紀錄且占用額度，等待明確結果。
- 待確認：結果未知，仍保留占用；查詢原退款，不改鍵重送。
- 已成功：同一交易增加已退金額並解除該筆占用，只結算一次。
- 已失敗：確定沒有成功退款才解除占用，不增加已退金額。

未知不能因本機等待太久就自行改成失敗。人工接手也不是自動釋放額度的理由。若供應商通知與終態衝突，先查核，不能無條件反轉帳務。

四個快照案例驗證成功、失敗、未知及重複成功通知。其中同一筆 300 元從占用轉成成功時，已退增加 300、占用減少 300，可退額度沒有因此增加；若結果未知，兩個欄位都不應改動。

這些快照是規格例子，不是後端狀態機實作。測試不會創建真正的交易鎖，也不能證明併發、故障恢復或回呼驗證正確。

13.5 六套工具各自如何處理這次變更

工具	本次決定	原因
Mermaid	新增退款受理圖	與原付款流程分開，避免流程過長
PlantUML	新增退款狀態圖	明確結果、占用、重複通知與未知處理
D2	原服務架構暫不改	假設不新增服務邊界；退款是既有角色的新操作，不自動新增元件
Graphviz	原依賴資料暫不改	既有 refund→payment 已存在；本次未提供新模組依賴
draw.io	原同步呼叫總覽暫不改	原圖明示退款另圖處理，本次不把更細流程塞回去
Excalidraw	保留原工作坊白板	原白板的「暫不處理」屬當時範圍；本章另記新增範圍，不回寫成歷史已決定

這是有理由的同步決策，不是宣稱六張圖都已改好。若真實需求變成新增退款服務、接上另一家供應商或改變模組呼叫，就必須重新檢查 D2、Graphviz、時序圖與可拖曳交接圖。

影響表 記錄原始來源與處理方式；沒有把服務呼叫箭頭當作模組依賴，也沒有因同名就合併支付角色。

13.6 讓錯誤案例真的被抓到

新增測試讀取六個額度情境與四個狀態快照，檢查金額上限與結果處理規則。它驗證資料是否符合本章明示公式，不是通用退款引擎。

```
python3 -m unittest discover -s tests -p 'test_refund_change.py' -v
make check-core
make check-full
```

在暫存資料中，刻意把未知結果的占用從 300 改成 0。測試確實失敗，指出 unknown or duplicate must not change ledger。用正常資料再跑，兩個測試方法通過。反例沒有寫回正式情境檔。

新增規格測試已加入 check-core，因此本次核心集合為六個方法，完整現有集合為十個。第 12 章的四個／八個結果仍是當時快照，不應回頭竄改成較新的數量。

REFUND_SCENARIOS 只供隔離副本故障注入，正式檢查請取消設定，以免意外驗證了別份資料。像第 11 章的 BOOK_SOURCE_ROOT 一樣，測試來源也是驗證證據的一部分。

13.7 圖能產生之後，再處理讀者看不清楚的問題

第一版 Mermaid 把受理、結果、對帳全部畫在同一張圖，成功與失敗的匯流線交叉且文字偏小。此次沒有刪掉規則，而是拆成受理流程與結果狀態兩張圖，再縮短交接標籤，重新輸出 SVG／PNG 並檢查。

最終兩張圖中文完整，未見明顯裁切或文字重疊。Mermaid 縮圖字仍偏小、留白不均；PlantUML 少數線條與標籤較接近，尚未達紙本最終排版驗收。請以原生來源修改後重產，不只改 PNG。

13.8 可以交接，不等於可以上線

交接包清單見 [README](#)，包含需求、來源、預覽、情境、測試與已知限制。正式產品仍需處理：

- 真實供應商募等期限、鍵範圍、退款／查詢語意及非同步通知驗證。
- 交易隔離、唯一約束、崩潰恢復、重放與競爭操作。
- 權限、濫用防護、客服流程、人工責任與對帳時限。
- 多幣別、金額精度、手續費、爭議款、稅務與業務政策。

本章沒有真實金流與資料庫測試，不能以 CI 綠燈代替上線審核。沒有新增 Release 或批准授權。

13.9 練習

1. 把未知結果的占用清零，觀察測試失敗。參考：未知不能釋放；應查詢原退款。
2. 同一退款鍵改成另一個金額。參考：內容衝突應拒絕；目前算術快照測試不涵蓋這個 API 行為，應另做實作測試。

3. 提出「新增獨立退款服務」。參考：這改變了原架構假設，重新走跨圖影響評估，不能只在原圖改標籤。
4. 把某次狀態快照通過測試，寫成「退款 API 已驗證」。參考：這是超出證據的聲明，需改為「本章規格情境斷言通過」。

本輪里程碑

本章完成後另已補齊第 1 至 3 章，並擴寫第 8 章；最新進度見 [章節索引](#)。13 章均有樣章，整本書的通稿、乾淨環境、排版與正式發布尚未完成。